

CONTEXT-NATIVE AGENT

Agent 重架构 迈向上下文原生智能

Agent = Environment + Context + Model

目 录

序言：余烬在剪枝	3
正文：欢迎来到上下文原生智能（Context-Native Agent）的世界	4
问题的由来	4
换一个主体	5
四个推论	5
被换掉的是解空间	6
具体实现	6
相对主流的优势	8
边界与未来	8
附录1：Harness耦合现象的思考 与 Context作为一等公民的必要性	9
附录2：半定量成本分析（已用真实账单锚定）	10
A2.0 计费模型与符号	10
A2.1 两套架构的成本来源构成	13
A2.2 两种钱、四个台阶	14
A2.3 基线不能“想压就压”	17
A2.4 做差：实测锚定与保守估计	19
A2.5 规模变大之后：三推一反	23
A2.6 没有计入账里的收益	24
A2.7 四条例外	26
A2.8 本节结论	27
附录3：引用、外部证据与对照	28
A3.0 约定	28
A3.1 条目	29
A3.2 相似架构对比与区分	32
A3.3 主张与外部证据	34
A3.4 已经发生过的坑，与本文对应的设计区分点	35
A3.5 至今没有人做过的对照	36
A3.6 口径与适用范围	37
A3.7 未采信与不可用清单	38
A3.8 一句话	39

序言：余烬在剪枝

上个月，Deepseek核弹连发，不管是掀起flash革命的Deepseek V4 Flash-0731，还是神鬼二象性的Deepseek V4 Pro-0813，至少都在大家伙心里留下了深刻的印象。更不要说面向AGI设计，万物皆插件的野心勃勃的Deepseek Harness了。对我这个👉小子来说，惊喜来的实在是太突然。我立刻动手体验了这个有趣的Harness框架，并测试了它的创造模式（虽然不出所料地把自己搞崩溃了）。然而噩耗来的总是那般突然，就在我兴奋地为我的个人小文件整理开源项目添砖加瓦的时候，梁文峰带着他的涨价来了。眼见着自己的费用蹭蹭上涨，上下文缓存那些钱也不由得让我心疼起来。可怜的余烬那时候还完全没有意识到，其实雷霆大思考才是上下文那么长的主因。于是我萌生了这样的想法：“要不我自己动手做个插件，把或成熟，或创新的上下文压缩机制缝进去，顺便美美收Github星星，投递一点大模型公司简历”。然而直到动手之前，我都还没意识到自己即将面临怎么样的“零花钱地狱”。

上下文管理是个被研究的十分透彻的领域，在通过免费快速的Deepseek网页快速模式Deeeeepseeek了一段时间后，我就笃定了自己已经对于Harness的上下文管理机制有了充分的了解。好！动手！结果自然是大败而归。

在dsh-price-less插件设计的初版，我进行了几个重大构想：**RAG/代码树为底层的语义化文件读取，基于意图的任务边界识别，任务边界与压力阈值自动压缩，提示词优化器。**

具体来说，任务，也就是Task，并非特指某种类型的任务，而是针对代码的某个结构/功能，具有文件读取特征的修改。根据这一定义，我们在边界区间压缩上下文能够最大程度避免上下文失效，降低重读率，同时拒绝为原本用不到那么多次，但是一直存在于缓存中的内容，大量重复计费哪怕很廉价的缓存计价。

而任务边界唯一的判定标准，就是用户输入。每轮任务内，全量的用户输入都会进入到“任务判别器”的上下文，再被组装上“项目整体目标”作为稳定前缀，确保任务边界判别的成本极低，压缩自动化且准确。而提示词优化器则更是巧妙复用，通过task前上下文，当前用户输入，与安装在Harness中的Skills列表按需取用，做到将用户的提示词扩写锚定，并且对于一些Skill能覆盖的操作给出最优解决方案，避免试错。RAG/代码树则是语义化，细粒度地读取文件，减少盲目全读和反复重读。

设想很美好，但是现实很骨感。因为实习要求在即，我不得不把RAG这种复杂内容搁置，移动到路线图中，全力开发边界识别。在这期间我经历了多次失败。包括机械判定完全不准确，embedding模型区分度极低，参数调优倾向于刀刀切，作弊获取奖励，评价标准难以综合多个指标。更不要说我作为夜猫子多次忘记梁文峰，足足烧掉了几十块的零花钱，最终才形成了上面那套能和提示词优化器复用缓存的成熟方案。

过程中还诞生了很多新的想法，包括工具剪切中间层等。通过对工具结果的保守通配，只保留有用信息，如编译器报错。但是在实测之后收益极低，最终被搁置。还有结论压缩，基于

和工具剪切的类似理念，将中间结论直接代替“思考-工具调用-再思考-结论”，嵌入上下文。但是实际上，提示词注入后，大模型很难按照要求给出剪切信号，机械剪切风险和收益不成正比，也被弃用了。还有压缩产物的结构，也让我殚精竭虑。通过任务和类型化子任务高度浓缩之前任务信息固然不错，但是模型申报原文作为热尾天天坐标不准组装出错也非常令我恼火。事实上，经过一些实际使用，有限的原文长度对于模型的复读阻止作用十分有限，现在已经在本地改成申报坐标了，原文什么的，自己多读几轮，至少还更精确吧。

庸庸碌碌忙活这么久，最大的成功还是边界压缩，给我气笑了。索性还是有点安慰，至少Kimi可能为真的面试题，竟然句句踩在我失败的embedding和机械路径上，总算是长舒了一口气。不对啊？人家零花钱比我多多了，要是真试出来更好的，那还是我亏麻了，气气气！

但是这样磕磕绊绊的探索，也让我产生了一个足够疯狂的想法。已知上下文剪枝非常困难，已知上下文积累速度极高，主要来自于模型调用读取的文件慢慢积累，慢慢因为长度和中段被遗忘，然后再次读取……

我们处在一个上下文不够用的环境下太久了，以至于我们一直在思考如何做减法……那有没有可能，上下文是应该自己长出来的呢？

正文：欢迎来到上下文原生智能（Context-Native Agent）的世界

问题的由来

所以，当我想到这一点的时候，一切就豁然开朗了。在过去，上下文极度紧缩，完全不够用的时代，我们花费了不知多少功夫，试图将海量内容塞在上下文里，通过各种索引RAG的方式确保模型能够瞻前顾后，将一切纳入考量，但是随着Deepseek家模型拉开百万上下文的普惠年代，30%上下就开始手动压缩成为了常态（目前平衡信息量与模型能力衰减的甜点位上下文长度在20-30%，不严谨，但是大概为社区主流观点^{[1][2][3]}；不过新近一份编码代理的白盒实测指出，这个甜点并不是一个稳定阈值^[44]），那么我们是否也该反思自己：“**上下文压缩与剪切，是否还是正确的上下文处理方式呢？**”

究其本源，我想我们要回归一个最基本的问题上：什么是Agent？在目前的社区主流与共识的说法中：“Agent = Harness + Model”。实际上Deepseek甚至将这个理念写入了他们的招聘要求中。在他们的实践中，甚至也出现了模型与Harness框架耦合才能解锁最高能力的现象。我对这一点其实并不持反对态度。虽然说在客观上这限制了用户API调用的选择范围，但是事实上这就好比人使用自己的手达成通用能力一样，这样的耦合也许有助于达成更高的能力上限。不过，当然更大的原因是Deepseek Harness是一款自由，开源的框架，并非是为了商业目的的强迫。

回到问题本身，Harness本身并不是什么简单的东西，尤其是学习Claude成熟经验后诞生的各种千奇百怪Harness框架。有的内置各种RAG，Skills，MCP，Tools，以Agent Loop为核心。将Harness视为Agent的组成部分诚然合理，但是这个框架太大，太重，让我们不知道如

何去管理。当我们预设了每一分上下文信息的必要性，那必然就要面临顺序，取舍与重读，还有困难的剪枝。历史和实践证明，我们总是习惯性地通过补丁来解决，引入更多的机制，更多的压缩，更多的坐标。可是我在这里想要问下一句，**有没有可能，上下文是从一颗纯净、被构建的前缀“种子”中生长出来的？**

换一个主体

当我们去审视大语言模型的本质，在它的视角里，并不存在什么所谓“Harness”，甚至就连思考过程都是依靠训练对齐的，它只是一遍遍地处理着语言输入的黑盒函数。**决定它输出状态的只有上下文与参数**，所有的工具调用，思考过程都是训练对齐的结果。那我想，我们应该拒绝以用户交互的对象为Agent主体的认知错位，事实上：**“Agent = Environment + Context + Model”**。

这句话反映的是两个层级：模型的状态只取决于参数与上下文；Agent的状态取决于环境 + 上下文 + 模型。

四个推论

那么这种理解方式能给我们带来什么启示？

- **推论1：** 在一个环境受控的Harness框架下，保存一个Agent所需的运行状态无需任何与当前Context无关内容，成本远低于“Agent = Harness + Model”的假设。
- **推论2：** 在同一个 Harness 框架下、且环境一致时，给定上下文就能立刻重建一个 Agent 状态。
- **推论3（普适）：** 上下文不该是 Loop Then Compress，而是按轮 **再生成 → 切片 → 合并 → 重载**（Regenerated, Sliced, Combined, Reloaded）。这一条不依赖环境：对纯对话系统（无文件系统、无进程、无副作用），它同样成立——上下文照样要重建、切分、合并、重载，只是没有需要维护的环境实体。
- **推论3'（Agent 特化）：** 当系统存在环境实体（代码、依赖、进程、端口）时，上述四步落到具体对象上即为——**环境维持共享、代码 fork、上下文 build**。

这三个推论共同指向的并非既有框架的补丁，而是**取消了一个约束**：内容离开上下文，不再等于失去内容。在旧框架里上下文是缓冲区，离开意味着内容丢失；而在新框架里上下文是投影，离开只是此刻无需^{[4][5]}。**压缩/剪枝与重建的全部差别，就藏在这个等号里。**

具体来说，**构建本身是能够被规划与预测的**；而当内容已经垒起来、后面的东西又压在它上面时，再回头抽掉其中一段的剪枝，就显得风险过大、成本太高。构建所做的像是CPU的缓存预取，只是这里的“预取”要模型自己判断、自己写下来：猜错了，也只是多背进来一点意义不大的背景信息，对任务落地几乎没有负面影响，在成本上也只是多一点——而且多出来的那部分落在最便宜的一档。可一旦构建正确，精准高效地组装了关键上下文，被省下的

就不只是背着的那部分：思考与工具调用的轮次会跟着收敛，输出与缓存也随之少下去，这还没算上干净上下文本身的能力增益（这两笔没有计进账里，见 A2.6）。然而这甚至还不是全部——在上下文的构建与组装里，**没有对与错，只有对多少与错多少；而错的代价不是损失，是少省了一部分**，于是不那么精准的装配，也能比现在的常见设计更省、更快。

最终，以上述的理念为核心，我们可以推导出下一代Agent范式，与其较为具体的工程实现。我称其为上下文原生智能（Context-Native Agent）。在这一架构下Harness的职责从管理Model转向管理Context与管理一个Agent切片运行所需的环境。Context与Harness复杂框架解耦，作为与模型唯一的交互通道被解耦出来，**成为Harness的一等公民**。

被换掉的是解空间

Context成为一等公民，意味着我们在文件治理，项目管理，操作系统与软件工程的成熟经验，均可以套用在管理Agent状态的切片上，实现高度透明，机械化，有保障的Agent协作，助力大模型在前AGI时代发挥能力上限。

然而，本文真正的价值不只在 Git 树与围绕它打造的这个系统上。Git 只是一个具体实现，粗糙、可替换，甚至可能是错的。真正被换掉的是 **问题的解空间**：

- 在 $\text{Agent} = \text{Harness} + \text{Model}$ 下，上下文是一段必须被管理的缓冲区。所有解法因此被逼向同一个方向——压缩、裁剪、摘要、检索——它们的共同点是**都要在信息量与长度之间做取舍**；而只要做取舍，就必然丢东西，就必然要赌。补丁套补丁、坐标套坐标，都是在这个解空间里打转。
- 在 $\text{Agent} = \text{Environment} + \text{Context} + \text{Model}$ 下，上下文是一份**可以从状态重建的投影**。这一步换掉的正是那个取舍：压缩是**删除**，不可逆；分区是**不构建**，可回取。既然上下文随时能从“环境 + 协议”重建，不构建它就**不再等于失去内容**。这个等号一解开，压缩就不再是唯一出路——而分区、委派、构建之所以在旧框架下显得奢侈，恰恰因为旧框架里没有“可重建”这个前提。

具体实现

以这一理念出发，一种上下文原生智能的具体实现可以以一种 AI native “Git tree” 为核心，其分为主Agent与子Agent，具有模型一致性。

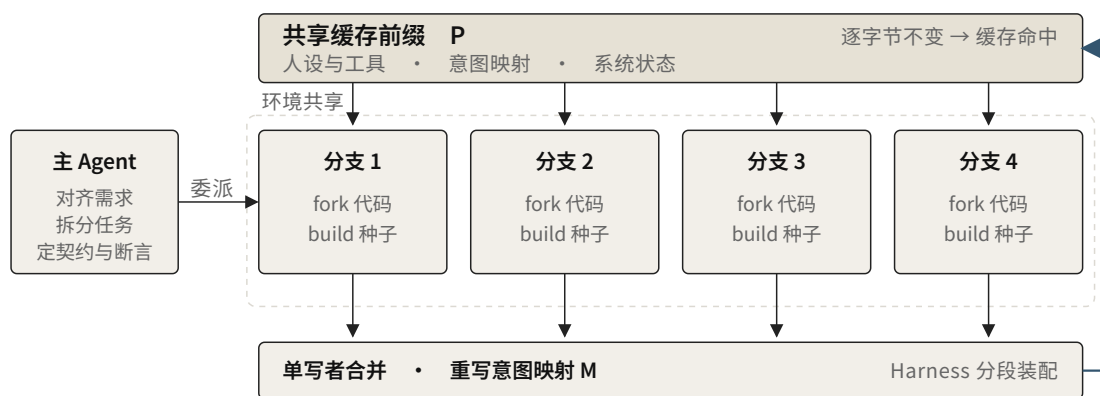


图3 一轮的动作：环境维持共享、代码 fork、上下文 build。全部子 Agent 回归后由主 Agent 单写者合并，并重写需要模型更新的那一段，其余各段由 Harness 从环境取出、分段装配成新版前缀。没变的字节照旧命中缓存；分支的上下文随分支消失，原文留在 git 里。数据源：正文“具体实现”一节。

对于任意一个项目，主Agent与子Agent拥有一个**共享缓存前缀**^{[6][7]}，其为当前系统版本下，必要的意图映射汇总，与必要的系统状态。当一个任务被委派时，主Agent负责与用户对齐需求，并对任务进行拆分，任务的拆分的具体原则以降低耦合度与提高子任务目标的内聚性为主，且对任务的规模进行控制，确保其上下文占用对齐模型能力甜点区间。

然后，其将为每个子Agent创建一个分支，具体而言：maintain environment, fork code, then build context as a seed。**每轮都要走一遍分叉、定契约与准备合并，这笔开销与任务多长无关**^[8]。每个子Agent的工作会作用于分支的虚拟文件上，以提交记录的形式落盘。

任务解耦完毕后，主agent将会运用其对于代码整体的理解，为每个子Agent委派上下文中预置的文件内容，然后由Harness进行**最大前缀组装**^[38]。且主Agent会制定断言与测试作为合并前必要验收标准。

任务过程中，子Agent的动作可能或主动或被动提交Signal，供主Agent进行环境观察与调度。也就是说，这一具体设计类似于一种响应式Agent调度框架。

全部子Agent回归后，由主Agent/委派Agent进行代码合并，取长补短，并进行最终测试与汇报，并同时会对系统状态、代码映射等进行更新——由主 Agent／委派 Agent 重写需要模型更新的部分，其余部分由 Harness 从环境**分别装配**成新的前缀^[9]。（这一步重写的是**变动的那一段**，而不是**整篇前缀**：一段短，整篇长，两者的价钱差一个量级，附录2会算这笔账。）这个新的前缀将会成为下一次执行时候的公共前缀，其作为全局理解对于之后的主Agent任务委派与子Agent的全局观起到指导作用，是衔尾蛇循环重启自己生命历程的步骤。

→ A2.4 这笔账怎么算

综上为我为上下文原生智能（Context-Native Agent）设计的一种Git Base的基础架构。其具有理解/执行模型解耦，过程透明可理解，并行效率高，各个Agent可复用现有单流程Agent上下文管理成熟经验，前缀缓存利用率高^{[10][11]}，且规避了上下文长度瓶颈，更是**从信息熵/上下文长度两方面确保了模型能力发挥稳定**。

相对主流的优势

具体来说其相对主流，定性可能具有的优势：

- 在上下文层面：每个子Agent只会被给定极其干净的 [Environmental View] + [Project Structure and Intention Mapping] + [Raw Files Assigned]，作为其生长出果实的“种子”。这样干净的上下文有助于模型立刻进行工作，减少读取的工具调用次数和探索信息的不可控，但因为只是种子，也保留了发挥空间与一定的不可控风险。在具体的工程实现上，如果Raw Files Assigned段太长，则考虑仅先放入公共前缀部分，后续内容通过一个特定工具按需取用^[9]。
- 在缓存利用层面：每个子Agent共享大量共同头，甚至Raw Files之前的内容可以与主Agent共享缓存前缀，等价于将所有只需要探索一次的内容的复利控制在了每个子Agent分区中，上下文累计只以子Agent结出的果实的形式回归主干，被吸收利用。这种共享有个前提：**装配时段序稳定**^[12]——前缀为了信息密度每轮都要更新一版，但段序一动不动，变动的只落在尾部，没变的部分逐字节相同。而分支那份上下文本身并不回归，它随分支一起消失，**消失的只是“曾经构建在上下文里”，不是内容**。而根据Git与软件工程的成熟经验，一次合并就意味着项目意图/功能结构的变更，此时主Agent上下文也是时候进行更新与重启了，最终做到每个目标实现时，上下文长度几乎不触及压缩阈值，每次变更只进行一次干净彻底的探索，不带着压缩后信息密度高却丢失细节的信息包袱。
→ A3.4 41 个字符的缓存崩塌
- 在透明度层面：主次分明，探索思考，需求对齐与实际落地解耦，交互均有signal可查，代码，架构与功能变更清晰可确认。
- 在模型适配性层面：可对齐每个模型所需求的最优上下文形状。
- 在模型能力层面：主要收益来自于信息熵的降低与上下文长度可控，中段迷失和能力下降被有效控制^{[13][34]}。

边界与未来

其有待实验验证的点在于任务规模与解耦设计对于成本与能力的具体影响。但是，随着Goal模式等交互模式的兴起，任务的复杂度在上升，且逐渐变得更加高层，与实现无关。因此，**传统的上下文累计思路必然会受到对齐需求与探索时期复利的反噬**，哪怕是压缩之后也一样。

而对于我们这套Git架构，越是随着任务的规模增大与目的的实现无关，就越可以被细化为边界清晰、数量可控、架构解耦的落地分支，并由主Agent一次生成一份稳定的、共享的、能回答“这个功能会落在哪些结构上”的项目意图映射与清晰边界的架构契约。最终规避并行写者 N^2 的冲突^{[14][15]}，因为那是在任务高度耦合的具体实现目标下的不理想情况。当然，具体方面的优化打磨，不易解耦的任务，我们的架构能以付出轻微额外成本为代价，即可轻松回退到主Agent/少分支结构上。

我相信 **build上下文会是上限更高，更符合未来需求与趋势的更优选择**。Git管理只是我结合软件工程经验给定的一个具体实现，只要思想存在，就一定会涌现出更多优秀设计。而当前Git架构下去，困难点在于虚拟文件的编译与测试、环境稳定性维护。

而在更加遥远的未来，当大模型的输出速度成为瓶颈的时候，缓存架构的优秀是否也意味着，首Token和平均Token都能大大提高^{[16][17]}？具有实际生产价值的模型规模上限得到扩充？不过这些都是当前架构成为主流顺带来的好处罢了，总之，它意在扭转还没有回头的迟滞当下，是面向未来以工程复杂度换取落地价值的优秀架构。现在的大模型架构革新，以收敛难度与炸炉风险换取能力上限，而 **Harness架构的革新成本与风险，可是要小得多，难道不值得探索吗？**

附录1：Harness耦合现象的思考 与 Context作为一等公民的必要性

Deepseek V4 Pro-0813在实际发布后饱受诟病，其被质疑为刷分模型，在除了极简模式 + 类linux之外环境下，均难以产生较好的结果。在上文中，我也曾经提到了关于Harness作为模型原生手脚的比喻，耦合以发挥模型能力上限。

而当我们放下个人视角，从模型训练的角度，Tools是否有泛化的必要与泛化的可能？

在我看来，格式化输出在训练上的对齐是十分必要的，但是模型在Tools的泛化上完全没有必要进行特殊训练。对Tools格式的理解完全可以被通用工具，代码调用覆盖，将参数与信息量浪费在对齐可能出现的不同自定义Tools上面是因噎废食。我们Harness构造的最终目的是能力上的通用，而非不同框架下的普适，就像我们不怎么需要去训练模型写什么Visual Basic代码一样。

而在数据构造的角度来说，我们给模型输入的构造数据，其涵盖的Tool类型也必然是给定有限，优先确定，通用的优秀组合。如Edit, Bash等。其主要目的就是聚焦Agent完成任务过程中，充当眼与手。如果要增加模型对于多样工具调用的适应性，为此降低其对通用普适，且已经足够达成通用能力的工具的调用能力，显然是得不偿失的。

那么，我们所需要面临的一点问题就是Harness框架作为产品的独立性与市场性又该如何保证？每一家的模型在自家Agent上都会默认胜出，那如何通过优秀的Harness设计抢占市场，制定标准，把握用户数据入口？

答案就在于Context作为与模型直接交互的中间层。现在的通用Harness在API层实现对模型的解耦，因此其不能对齐模型的训练数据与偏好环境。真正行之有效的适配方式，是在上下文组装环境为每个模型进行分野。不同的工具调用对于模型的形状各不相同，对应着模型的原生“肢体”。这样的一套通用Agent架构，才能实现对于同一模型，同一发挥下的“可比性”。

附录2：半定量成本分析（已用真实账单锚定）

本节不做实验，只做账。

所谓“半定量”，是只做到**量级正确、方向可靠**，不去追一个精确到小数点的数字。

账本上守三条规矩：

- 1. 每个参数都取对结论不利的那一侧。
- 2. 能实测的用实测值；不能实测的才取保守值，并且标明它是保守值。
- 3. 算完回头查一遍：这些“保守”取法，是不是真的都在对我们不利？

第三条不是客套。查完之后，绝大多数取法确实偏在不利的一侧，**只有四条例外**，单独列在 A2.7——它们决定了结论能走多远。

参数全部来自本地归档里三条**从未装过任何上下文处理插件**的干净原始记录：合计 9,023 步、363 小时、54 次原生压缩，逐次调用的计费数据完整保留。本节凡是写“实测”的地方，都指向它们。

还有一件事要在开头说清，因为后面每个数字的读法都由它决定：**上下文是每一轮从 0 构建出来的**，“一份内容不在上下文里”是默认状态，要它出现才需要有人决定、才需要付钱。四个台阶（A2.2）正是按这个前提排的。

再把这一节量的是什么说清楚：它是本文那处具体实现（Git 树）的账。参数逐项在 A2.4 交代，每一项都取对结论不利的一侧；换一处实现、换一组参数，数字会变，**而四个台阶、三个判据、有效残留这几样东西的形状不变**。所以下面的结论属于这处实现，不属于“每轮重建”这件事本身。

A2.0 计费模型与符号

三档价格。全文按 DeepSeek V4.1 Flash 低谷时段计价^{[18][19]}：

计费项	单价	什么时候按这一档计
缓存命中	¥0.02 / 百万 token	这段内容没变、被缓存复用
缓存未命中	¥1 / 百万 token	这段内容是新的，或者缓存已经失效
输出	¥4 / 百万 token	模型写出来的每一个 token

两个比值决定了整笔账的形状：**未命中是命中的 50 倍，输出是命中的 200 倍**。换成一句人话：

模型每写 1 个 token 的钱，等于把 200 个 token 背着走一步。

这句话不是修辞，它的量级由实测的用量撑住。把三条实测会话合计 9,023 步的 token 量按上面这张价格表算一遍^[20]：

钱花在哪	token 量	花费	占比
背着（内容没变，命中复用）	1,817.1 M	¥36.34	38.5%
第一次读新内容（未命中）	20.5 M	¥20.50	21.7%
写（模型输出）	9.41 M	¥37.64	39.8%

平均每步：背 203,660 个、写 1,043 个。写量只有背量的 0.51%，花的钱却是背的 1.04 倍——这就是 ¥4 与 ¥0.02 那 200 倍价差落在真实用量上的结果。

这张表还澄清了一个容易滑过去的说法。按单价，背着确实极便宜；但真实任务里背的量是写量的 200 倍，于是两者在账单上各占约四成。所以准确的说法不是“背着不要钱”，而是：

背着便宜，正好被它庞大的数量抵消掉了；而写只用了 0.5% 的数量，就换来了同等的账单。

所以这两笔钱哪一笔更要紧，不能先验地断定——这也正是下一节必须一条一条算的原因。

而这个差别，本质上是有损与无损的差别：

	重置时要写下来什么	结果
基线（有损压缩）	内容：把这个区间里“以后可能还要用”的东西概括成摘要	摘要必须承载语义，所以写得长
本架构（无损卸载）	变动的映射：只写需要模型更新的那一段（意图映射、系统状态），原文仍然在 git 里	映射只是索引，所以写得短；而且原文本来就不必待在上下文里，要用的时候读一次就行

注意“写下来多少”和“上下文里占多少”是两件事：摘要和映射都要占据上下文（这部分只是被读，便宜），但只有“写出来”的那一步按输出价计费。

符号。后面会用到的每个符号，这里先用白话说清楚。全文只按 token 计数；token 是模型读写和计费的最小单位，大约相当于半个到三分之二个汉字。

符号	白话含义
窗口 W	一个模型一次请求最多能装多少 token。本文一律按 100 万 token ($W = 1\text{M}$) 举例。
压缩	装不下了，就让模型把已有内容读一遍、写一份更短的摘要，再用这份摘要替换掉原来那一段。
残留 L_0	一轮刚开始、上下文刚构建完的那一刻，里面装了多少 token。基线的残留是 C_c （摘要 + 保留的尾部），本架构的残留是 $P + s$ （前缀 + 切片）。

符号	白话含义
触发点 θ	上下文涨到“窗口的百分之几”的时候触发一次压缩。 $\theta = 50\%$ 就是“装到一半就压”。
步数 T	模型的一次“思考 + 动作”算一步，整个任务一共 T 步。
每步新增 g	每一步平均往上下文里新增多少 token。
前缀 P	所有分支、所有轮次都用、而且内容稳定的那一段：人设与工具清单、项目结构说明、意图映射（哪个功能大致落在哪些结构上）、必要的系统状态。注意 P 是它在上下文里占多少 token，是一个“读”的量。
更新量 M	主 Agent 为了更新前缀，真正写出来多少 token。因为只写需要模型更新的那一段， M 通常远小于 P 。
构建	每一轮开始时，Harness 照当时的环境把这一轮要用的内容装配成上下文。它不花输出钱，只让模型多读 P 个 token。
切片 s	只发给某一个分支的那部分文件内容。
不在上下文里（默认）	一份内容没被构建进这一轮上下文的状态。这是默认状态：不花钱，也不需要谁做决定。
边界	一轮结束、一个分支退出或整个任务收尾时，那一轮的上下文整体消失。原文都在 git 里，消失的不是内容。
摘要长度 σ	基线每压缩一次，模型要写出来多少 token（摘要 + 思考）。
命中价 p_h	内容没变、被缓存复用时，读一个 token 的单价。本文取 ¥0.02/M。
未命中价 p_m	内容变了、缓存作废、必须重新读一遍时的单价。本文取 ¥1/M。
输出价 p_o	模型写出来的 token 的单价。本文取 ¥4/M。
重发现 ρ	被摘要丢掉的信息，后来又被用到，只能重新推导一遍。每一步平均要重新推导的量记作 ρ 。
压缩间隔 k_b	基线的两次压缩之间隔多少步。
轮长 L	本架构里，一个分支走满一轮需要多少步。
轮数 R	本架构一共分多少轮。每合并一次、并重建一次前缀，算一轮。注意一轮里有 N 个分支并行，所以按总调用数摊算是 $R = T/(N \cdot L)$ ，不是 T/L 。
分支数 N	一轮里同时开多少个分支。
协调开销 ϵ	本架构比基线多出来的步数比例（定契约、委派、写断言这些步）。
每次重置成本 E	一次“重置”要花的钱。基线是 E_b ，本架构是 E_d 。
有效残留 R_b, R_d	把“每次重置要花的钱”折算成“相当于常态多背多少 token”，再添加到残留上，得到一个两边可比的量。有效残留小的那一方更省。
差额 Δ	两套架构每一步平均占用的 token 数之差。
前缀变动量 ΔP	新版前缀里真正变了的那一段有多少 token。段序稳定时它落在前缀尾部，前半段照旧命中缓存。
重发现盈亏平衡点 ρ^*	基线的重发现量 ρ 涨到多少，才会把本架构的成本优势吃光。

A2.1 两套架构的成本来源构成

基线（传统累积 + 边界压缩）的成本来源

#	类别	来源	具体是什么	随什么增长
1	背着	过时内容被反复重读	早就没用了的中间结果还留在上下文里，后面每一步都要重新读它一遍	T^2 （完全不压缩时）／ $T \cdot k_b$ （压缩后）
2	重置	压缩把缓存打掉	压缩会把替换点之后的上下文全部重写，缓存作废，之后只能按 未命中价 重新读	$(T/k_b) \times C_c$ （未命中价）
3	重置	写摘要	每压缩一次，模型就要写一份摘要，还要先想一遍， 这是输出价	$(T/k_b) \times \sigma$
4	重置	重发现	摘要丢掉的细节之后又被用到，只能重新推导一遍， 同样是输出价，而且不封顶	$\rho \cdot T$
5	背着	压缩这个动作要读输入	压缩时必须把被替换掉的那一段重新读一遍（多数是命中价）	$(T/k_b) \times \text{区间长度}$
6	背着	探索不受控	上下文越长，越容易在中段丢信息，模型乱找路的概率越高	表现为 T 和 ρ 都变大

本架构的成本来源

#	类别	来源	具体是什么	随什么增长
1	固定	分叉	每一轮做一次 git 操作，给分支切一份代码	R ，固定、极小
2	重置	定契约	事先说好每个分支要产出什么、边界在哪里。它在上下文状态上与合并只差一条提交记录，因此和合并算同一件事	与合并同项
3	重置	合并与解冲突	两个分支改到同一批文件时要解开冲突，成本由这种重叠的程度决定	N^2c
4	重置	更新前缀	主 Agent 只重写需要模型更新的那一段 （意图映射等），其余各段（人设与工具清单、系统状态）由 Harness 从环境取出， 分段装配成新一版前缀 。写出来的量是更新量 M ，按输出价；装配出来的 P 是被“读”的，走命中价， 只有每版变动的那一段 ΔP 才付一次未命中	$R \times M$ （输出价） + $R \times \Delta P$ （未命中价，每版一次）
5	背着	回归测试	改完跑一遍测试，确认没有弄坏原有功能	与基线相同，两边抵消

关键观察：这套账里只有两种钱——“背着”和“重置”。“背着”单价极便宜（¥0.02/M），“重置”极贵（¥4/M，是前者的 200 倍）。基线的重置有三项（2、3、4），其中重发现（第 4 项）不封顶；本架构的重置只有一项，而且写的是**变动的那几段、不是整篇内容**，所以它的量级是 $R \times M$ 加上每版一次的读，而不是 $R \times P$ 。

还有第三笔钱：第一次读新内容，实测里它占 21.7%。每一步产生的工具结果、用户消息，都是全新内容，第一次读必然未命中，之后才转为命中。它在两套架构里都存在，因此**做差时会抵消掉**，不进结论；但它不是零——“背着 38.5% + 重置 2%”并不是全部账单。

还有一件比这两笔钱更根本的事：本架构省下的钱，大头不是“少写”，而是“根本没构建”。基线把一份内容构建进上下文之后，想让它出来，只有压缩一条路。本架构里，内容默认就不在上下文里——要它出现，得先有人决定构建它。需要判断的问题于是从“该丢什么”变成了“该构建什么”，而后者是照着映射就能核对的事。下一节把这条差别折成四个价码来看。

A2.2 两种钱、四个台阶

这套账里只有两种钱：背着（内容装在上下文里，每一步都要被读一次）与重置（上下文换一版时，我们必须写下一点什么——基线写摘要，我们写变动的那一段）。

但往下看之前，得先接受一个前提——后面每个数字的读法都由它决定：

上下文不是攒出来的，是每一轮从 0 构建出来的。

模型自己不带记忆。每一轮递过去的上下文，都是照着当时的环境重新构建一份，而不是把前一轮接着往下续。所以在这套架构里，“一份内容不在上下文里”是默认状态——不需要谁做决定、不需要动作、不花钱；反过来，“它在上下文里”才是那个需要理由的状态：得有人决定把它构建进来，付一次它头一回被读的钱，然后在此后的每一步继续为它付钱。

这个前提一摆出来，两条路线的差别就不再是“谁删得更聪明”，而是“谁需要删”。基线把内容构建进去之后，想让它出来只有压缩一条路：一次有损的、必须做判断的、要付三笔钱的动作。本架构里没有这个动作——内容默认就不在里面，要用才构建，用完回到默认。序言里记的那三次翻车（工具剪切收益极低、结论压缩拿不到模型的剪切信号、压缩产物的坐标不准导致组装出错），全都是“做减法”时翻的车；减法在这套架构里根本不存在。

一份内容在这一生里，会在四个不同的价码上各停一停，而它们的单价差着两个数量级：

台阶	这份内容在做什么	单价	折合“背着”多少步	属于哪种钱
一	被写下来（写摘要与写更新同档）	¥4 / M	200 步	重置
二	第一次被构建进上下文（构建过又离开、之后再取回来一次，也是这一档）	¥1 / M	50 步	背着（未命中）
三	此后每一步都被读一次	¥0.02 / M	1 步	背着（命中）
四	根本没被构建进去（原文留在 git 里，要用的时候读一次）	0	0 步	不付

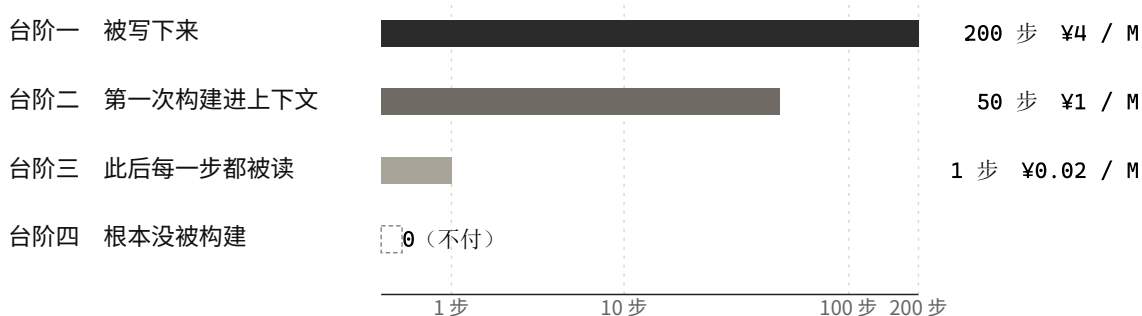


图 1 一份内容的四种价钱：把单价换算成“背着它走多少步”。数据源：附录2 A2.2（横轴对数刻度；台阶四为 0，用空心方块标记）。

第四列是这张表最有用的一列——它把“贵不贵”换算成了同一个尺子。**写一次的钱，等于把它背 200 步；构建一次的钱，等于把它背 50 步。**于是对已经构建进来的每一份内容，该留还是该走只看一件事：**它下一次被用到，大概在多少步之后。**五十步以内还会用到，就让它留着（继续待在台阶三）；下一次用到在五十步以外，就让它走（回到台阶四），到要用的时候再构建（台阶二）——只有那种每一步都得被参考的东西，才配一直停在台阶三。

只要一份内容还留在上下文里，模型每一步就必然要读它一次——台阶三收的就是这笔钱。所以“不构建”不是省钱的小技巧，它是一条判据：

对每一份内容挨个问一句：它值不值得此后每一步都被读一次？

值，就一直背着（比如那张“这个功能会落在哪些结构上”的意图映射）；不值，就让它用完之后回到“不在里面”的默认态，要用的时候再构建。

我们的纪律只有两句，它们管的是两件不同的事：

- **第一句管“写”和“构建”：每一份内容，台阶一只付一次、台阶二只付一次。**一份原文尽量不要这一轮写一遍、构建一遍，下一轮又写一遍、构建一遍。这也是“把它写进前缀”比“反复按需读取”划算的原因——前缀每一轮都装配在同一个位置、逐字节不变，于是它只在第一次（或它真正变动的那一次）付台阶二，之后每一轮都直接命中。
- **第二句管“留”：构建完就让它走，除非它值得此后每一步都被读一次。**

“一直背着”和“只付一次的未命中钱”因此并不矛盾：一直背着的是那几样每一步都要看的東西（前缀），其余的只构建一次。而基线的病根，正在于这两句它一句也做不到——每压一次就把新残留整段重读一遍（台阶二付了又付），同时把一大堆积压内容一直背到触发点（台阶三也一直付）。实测里换轮时的缓存失效（占全部未命中 token 的 34%–43%），就是前一句的账单。

先算“背着多少”。上下文在两次重置之间，是从“残留”一路涨到“触发点”的。如果每一步新增的量大致固定，那它就是一条直线；而一条直线在这一段里的平均值，正好等于两端的中间值：

$$\bar{L} = \frac{L_0 + \theta W}{2}$$

符号	白话含义	基线	本架构
θ	触发重置时，上下文占窗口的比例	θ_b : 实测中位 31% (见 A2.3)	$\theta_d \approx 20\% \sim 30\%$
L_0	一轮刚开始、刚构建完时的残留长度	C_c (摘要 + 保留的尾部)	$P + s$ (前缀 + 切片)
$\bar{L}$	每一步平均带着多少 token	$(C_c + \theta_b W)/2$	$(P + s + \theta_d W)/2$

因为两次重置之间是直线增长，这个式子不含可调参数：它是算出来的，不是拟合出来的。

实测校验：这个式子对得上量级。 干净会话实测每步平均占用 208,853，式子算 $(15.2k + 310k)/2 = 162.6k$ ，低估约 28%——因为增长并不均匀，用户换轮时也不重置时钟。偏差在两边以同样方式存在，不改变比值，只让本节给出的**绝对金额整体偏低**。

于是两边每一步“背着”的差额是

$$\Delta = \frac{(\theta_b - \theta_d)W + (C_c - P - s)}{2}$$

整个任务里“背着”这一侧的差额，换算成钱就是 $T \cdot \Delta \cdot p_h$ 。

再算“重置一次多少钱”。

基线重置一次，要写一份摘要，还要把压缩后新拼出来的上下文第一次读一遍：

$$E_b = \sigma \cdot p_o + C_c \cdot p_m$$

其中第一项是**写摘要**（输出价），第二项是**新内容第一次读**（未命中价）。第二项只按**压缩后真正要重读的那一段**计：压完保留的残留里有一部分字节仍在缓存中——实测九次压缩后首次调用的未命中均值是 **9.2k**，而残留的中位是 15.3k。

本架构重置一次，要写一份更新（只写变动的那一段），还要让分支把这一版新前缀读一遍——**未命中只付一次，不按分支数付**：

$$E_d = M \cdot p_o + \Delta P \cdot p_m + (P - \Delta P) \cdot p_h$$

第一项是**写更新**（输出价）。第二项要拆成两半，因为**前缀是分段装配出来的**——模型只重写该由模型更新的那一段，其余各段由 Harness 从环境取出：

- 被重写的那一段 ΔP ，缓存作废，按未命中价付；
- 没变的那些段 $P - \Delta P$ ，字节没动，**照旧命中缓存**，按命中价付。

只要装配时段序稳定（本设计要求的正是“重写不重排”），没变的部分就一直躺在缓存里。而且这一版前缀只在**第一个分支的调用**上付这一笔，之后所有分支、所有轮内步骤全部命中——所以它按“前缀版本数”计，**不按分支数计**。

基准档在这里取最不利的一侧：整段前缀按未命中算 ($\Delta P = P$)。真实机制是只有变了的那一段 ΔP 失去缓存、没变的那段照样命中——本设计的“重写不重排”正是为后者写的；两者的差额见 A2.7。

A2.3 基线不能“想压就压”

基线的触发点 θ_b 不是一个可以随便拧的旋钮。它要压低水位，手段只有压缩，而压缩是花钱的动作。两次压缩之间只隔

$$k_b = \frac{\theta_b W - C_c}{g}$$

步。触发点压得越低，这个间隔越短，压缩次数涨得越快。于是基线手上有两笔方向相反的钱：

基线把触发点压低	哪一笔变便宜	哪一笔变贵
θ_b 下降	背着的钱按 θ_b 线性减少	重置的钱按 $1/(\theta_b W - C_c)$ 上升

两笔一平衡，就得到基线自己会选的那个最优触发点：

$$\theta_b^* W = C_c + \sqrt{\frac{2gE_b}{p_h}}$$

代回去，基线能压到的最低总成本是：

$$C_b^* = T \cdot p_h \cdot R_b, \quad R_b = C_c + \sqrt{\frac{2gE_b}{p_h}}$$

这里的 R_b 读作“有效残留”——它把“每次重置要花的钱”折算成了“相当于常态多背多少 token”。

现在把实测值代进去。用干净会话量出来的 $g = 1,460$ 、 $C_c = 15.2\text{k}$ 、 $E_b \approx \text{¥}0.029$ ，基线自己的成本最优触发点是

$$\theta_b^* = \frac{C_c + \sqrt{2gE_b/p_h}}{W} \approx 8\%$$

也就是每 45 步压一次、一个两千步的任务压 46 次。而实测是这样的——干净会话里每一次压缩我全取出来了，全部由手动触发，丢弃量由 Harness 自己报出：

触发时占用	丢弃 token	压完保留	丢/留
23.5%	165,397	14,585	11.3×
29.3%	206,478	15,489	13.3×
52.4%	433,498	14,427	30.0×
35.2%	242,800	15,011	16.2×

触发时占用	丢弃 token	压完保留	丢/留
32.5%	244,932	13,834	17.7×
44.7%	333,979	17,933	18.6×
5.9%	27,203	15,334	1.8×
20.2%	115,939	15,486	7.5×
30.9%	178,092	15,908	11.2×

触发占用中位 **30.9%**；九次总共丢掉 194.8 万 token，每次只留 1.5 万。

注意唯一那一次 5.9% 的压缩：丢/留比掉到了 1.8×。占用已经很低时人就不再压了——不是不想省钱，而是再压就真的没有东西可压了。

8% 这个“成本最优”成立的前提，是把 $\rho = 0$ 和“丢掉的东西一文不值”两条假设推到极致。它数学上成立，工程上等于算出了一条“把球砸向地板再弹进篮框”的轨迹——它把任务继续执行所必需的那些上下文当成了零。

九次里，那个能看见任务的人九次都不同意这个判断：他停手的位置，中位数是 30.9%。

按实测，这句话应该这样写：

20%–30% 是能力上的要求（实测：人停手的位置中位 30.9%）；而只有有损压缩这一件工具时，钱算出来的均衡在 8%。两者的差，就是“压缩税”的实测大小——写摘要、缓存作废后的重读、以及丢掉的细节日后被重新推导，这三笔合起来的一道税，它大到没有人敢去领。

本架构的式子形状与基线完全一样。我们也要重置，重置成本是 E_d （见 A2.2）。它由三件事决定，每一件都不是估计，而是这套设计本身决定的：

1. 写的是变动的那一段，不是整篇内容。 M 是模型每轮写出来的更新量， σ 是摘要量。摘要必须承载语义（“这段区间里以后还要用的东西是什么”），映射只承载结构（“这个功能落在哪些结构上”）。不在上下文里的那些内容，原文还在 git 里，所以不需要把它们写进上下文。
2. 一次重置服务 N 个分支。一轮里 N 个分支并行推进，而合并与重建前缀在这一轮结束时只做一次。按总调用数摊算，我们的重置频率约为基线的 $1/N$ 。
3. 未命中只付变动的那一段。前缀没变的部分照旧命中缓存（这一条的分量由 A2.7 算清楚）。

在实测参数下，我们单次重置反而更贵：因为我们的残留（ $P + s = 60\text{k}$ ）是基线残留（ $C_c = 15.2\text{k}$ ）的四倍，而每换一版前缀就要把它读一遍——**单次重置 ¥0.060，比基线的**

¥0.029 贵一倍。 我们赢回来的是**频率**：基线每 202 步重置一次，我们每 384 步 ($N \cdot L = 4 \times 96$) 一次。两边相乘之后，重置这一侧基本打平 (¥0.32 对 ¥0.30)。

所以这套账的胜负不在重置，而在“背着”。 而我们“背着”更少的唯一原因，是我们能合法地站在更低的水位上——这句话在 A2.4、A2.6、A2.7 里会反复出现，因为它就是全部结论。

最优触发点满足同一个形状的式子：

$$\theta_d^* W = (P + s) + \sqrt{\frac{2gE_d}{N p_h}}$$

于是两套架构被放进了同一个式子里：谁的有效残留更小，谁就更省。

$$R_d = (P + s) + \sqrt{\frac{2gE_d}{N p_h}} \quad \text{对比} \quad R_b = C_c + \sqrt{\frac{2gE_b}{p_h}}$$

A2.4 做差：实测锚定与保守估计

本节把每个参数都标出来源和方向：能从记录里量出来的就量，量不出来的取保守值，并且说明这个方向为什么算是保守。

能实测的（从干净会话里量出来的）：

量	取值	从哪来	说明
W	100 万 token	请求头	前提
T	2,068 步	干净会话	任务全程
g	1,460 / 步	实测均值（中位 703）	增长并不均匀，均值更能代表累积量
θ_b	31%	九次人工压缩的中位	人停手的位置
C_c	15.2k	实测	摘要 + 保留的尾部
压缩后重读段	9.2k	九次压缩后首次调用的未命中均值（实测）	残留中位 15.3k 里仍在缓存的那部分不计
σ	5k	摘要正文 3-8k	写出去的量
E_b	¥0.029	由 σ 、 C_c 与价格算出	单次压缩的钱
k_b	202 步 （实测 230）	由 θ_b 、 C_c 、 g 算出	两次压缩之间的间隔
缓存命中率	98.6% - 99.7%	实测	整套账的前提，成立

量不出来、只能取的（逐条标出这个方向偏哪边）：

量	取法	取这个方向的理由	偏哪边
价格	取低谷价	缓存越便宜，“背着”那一侧的优势越小	偏不利（保守）
$P + s$	取大（ $P = 20k$ 、 $s = 40k$ ，合计 60k），并且假设切片整段预先构建	本架构残留得越多，它越重；按需取用本来能让切片的一部分根本不进上下文（见 A2.6 第 6 条）	偏不利（保守）
M	取大（10k）	更新量越大，我们的重置越贵。现实里要写的只有变动的那一段，量级是一步的输出（实测每步输出 833–1,133 token）	偏不利（保守），且偏得极重
ΔP	取极端（= P ：整段前缀按未命中）	我们的重置因此最贵	偏不利，但越过了真实机制（见 A2.7）
ρ	取 0	重发现完全不计——白送基线一笔	偏不利（保守）
ε	取 0	本架构多出来的协调步完全不计	偏不利（保守）
合并冲突 N^2c	不入账（只写进 A2.1 的成本表）	每轮真实存在	偏不利（保守）
质量与能力收益	一律设为 0	见 A2.6	偏不利（保守）
并行省下来的时间	不计入结论	不拿时间上的好处去补贴成本上的主张	偏不利（保守）
N	取 4	把“设计被正确使用”当成了既定事实	偏有利（不保守）：实测 $N = 1$
θ_d	取 20%	假定我们能在很低的水位上正常工作	偏有利（不保守）：这是能力假设（见 A2.7）

表中 $N = 4$ 与 $\theta_d = 20\%$ 两条偏有利，另有 $\Delta P = P$ 一条越过了真实机制；此外 E_b 里的压缩间隔取了公式值 202 步而不是实测的 230 步，方向也偏有利。这四条例外各自把结论推到哪一步，都在 A2.7。

$\theta_d = 20\%$ 偏向我们自己：不受能力约束时，我们自己的成本最优会压到 8%–11%（取决于更新量与前缀计价）。基线的钱最优也落在 8% 一带，但那一档对基线不是一个能干活的工作点（见 A2.7 第二条）。为了让读者能自己判断，下面同时给出 $\theta_d = 31\%$ 的算法核对结果；这一条的完整讨论在 A2.7。

先算中间量：

中间量	怎么算	结果
基线每次重置 E_b	$5k \times \text{¥}4/\text{M} + 9.2k \times \text{¥}1/\text{M}$	¥0.029

中间量	怎么算	结果
基线的压缩间隔 k_b	$(310k - 15.2k)/1,460$	202 步 （实测 230；本节取 202，见 A2.7 例外四）
本架构每次重置 E_d	$10k \times ¥4/M + 20k \times ¥1/M$ （前缀整段按未命中）	¥0.060
本架构每次重置 E'_d	$10k \times ¥4/M + 20k \times ¥0.02/M$ （前缀走命中）	¥0.040
本架构的轮长 L	$(200k - 60k)/1,460$	96 步
本架构的轮数 R	$T/(N \cdot L) = 2,068/(4 \times 96)$	5.4 轮

然后是两套架构各自的账（一次 2,068 步的任务）：

	背着	重置	合计
基线，停在实测的 31%	¥6.72	¥0.30	¥7.02
基线，撞 DeepSeek Harness (dsh) ^[40] 默认阈值 80%（完全不管控）	¥16.86	¥0.11	¥16.97
基线，压到它自己的钱最优 8%	¥1.98	¥1.35	¥3.33
本架构，20%、4 分支、更新量按保守的 10k	¥5.38	¥0.32	¥5.70
本架构，同上但前缀按命中计价	¥5.38	¥0.22	¥5.59

结论：对实测的真实基线（人停手在 31%），省约 19%；对完全不管控、一路撞到 80% 默认阈值的用法，省约 66%。

对照“基线自己的钱最优”（8%）：那时它 ¥3.33，我们 ¥5.70，我们贵 71%。而那条路要求它每 45 步压一次、一个任务压 46 次——实测里人压了 9 次。这不是一个“它随时可以选”的选项，这是 A2.3 里那条“砸地板进篮框”的轨迹。

把几个最要紧的假设单独拎出来看：

情形	我们的成本	对真实基线 ¥7.02	对不管控基线 ¥16.97
基准：20%、更新 10k、4 分支	¥5.70	省 19%	省 66%
两边都按在 31%（算法核对，不是我们会选的位置）	¥7.83	贵 12%	省 54%
不并行（ $N = 1$ ），其余同基准	¥6.67	省 5%	省 61%
不并行 且 前缀按命中计价	¥6.25	省 11%	省 63%
更新 2k + 前缀按命中	¥5.42	省 23%	省 68%
双重悲观：整篇手写 20k 且不并行	¥7.53	贵 7%	省 56%

两条读法：

- 只有在“模型整篇手写前缀”并且“完全不并行”同时成立时，我们才会输给真实基线——而这两件事正是这套设计本身否定的。（ $N = 1$ 单独发生时不构成失败：基准档下仍

省 5%，前缀按命中计时省 11%。）

2. 两边都按在 31% 时，我们反而贵 12%。这不是算错，是这套账的结构决定的：我们的残留 ($P + s = 60k$) 是基线残留 ($C_c = 15.2k$) 的四倍。本架构的钱，全部赚在“能在低水位上干活”这一件事上。而这一行是算法核对，不是两套架构各自会在的位置——低水位对基线不可达，高水位对我们没必要（理由在 A2.7 第二条）。

胜负由什么决定。把两边并排，只有三个量能动。

一、我们自己的残留 $P + s$ 有一个明确上限。令两边的有效残留相等，解出我们的残留最多可以多大：

$$P + s < C_c + \sqrt{\frac{2gE_b}{p_h}} - \sqrt{\frac{2gE_d}{N p_h}}$$

更新量 M	前缀计价	残留上限 $P + s$
2k	命中	63k
10k	命中	42k
10k	整段未命中	34k

读法：只要“前缀 + 切片”不超过 63k 我们就赢；本节保守取的 60k 刚好卡在线内，只剩 3k 余量。这条线不是装饰——它意味着“前缀能不能比摘要更省地表达全局”是本架构的生死线，而不是一个可以无限膨胀的容器。

二、更新量 M 的盈亏平衡点约 2.8k。当前缀按缓存命中计价，只要每轮写出的更新不超过 2.8k，我们就赢；而它的现实量级就是一步的输出（实测每步输出 833–1,133 token）。但如果前缀整段都按未命中计，这道式子无解——那时 E_d 里光“把前缀读一遍”就是 ¥0.02，已经超过全部预算 ¥0.0115，无论更新量多小我们都赢不了。这一笔在 A2.7 里单独交代。

三、重发现 ρ 的盈亏平衡点是 160–194 token/步。按实测参数，我们领先真实基线约 ¥0.00064–0.00078 每步，而 ρ 走输出价，于是

$$\rho^* = \frac{0.00064 \sim 0.00078}{p_o} \approx 160 \sim 194$$

（单位：token / 步。这正是那笔在账单上从来不会单独列出来的钱——因为它伪装成了“模型正在思考”。）

实测里压缩之后的追赶峰值到过 1,553 与 4,983 token/步（九次压缩中有六次测不出追赶，两次很明显）。所以 ρ 不是一个“符号对不对”的问题，而是一个量级相当的真实风险：只要基线平均每步多花 200 个 token 重新推导，那 19% 就被吃光了。

N 与成本的关系最容易被高估。按前缀命中计， N 只在更新量 M 大时才重要： $M = 2k$ 时， N 从 1 涨到 4 只带来 2 个百分点的差别； $M = 10k$ 时带来 9 个百分点。开分支的理由因此落在能力、隔离与可回滚上，而不是成本。

于是判据变成三个具体的、可以被否掉的问题：

- 一、每轮更新前缀时，模型真正“写出来”的是变动的那一段（不超过 2.8k token），还是整篇前缀（20k token）？

二、装配前缀时，段序是否稳定到“没变的那半段能一直命中缓存”？（这一条决定生死，见 A2.7）

三、“前缀 + 切片”能不能压在 63k 以内？

第一条正是本文序言里已经走过的路——从“申报原文”改成“申报坐标”。第二条的分量最容易被低估。第三条是规模的暴露面。

A2.5 规模变大之后：三推一反

A2.3 的两个闭式里有两个容易被忽略的性质，先说清楚，因为它决定了趋势该怎么谈：

- 任务长度 T 只是一个共同乘数。两边都随 T 线性放大，所以 T 不改变胜负方向，只放大结论。“任务会变长”本身不是论据——这一点必须先说清楚，否则会给人“我们是靠任务长赢的”错觉。
- 窗口 W 不进入最优总成本。因为 W 变大时最优触发点会跟着往下走（ θ^* 是解出来的，不是定死的），两边的最优总成本里都没有 W 。“百万窗口”对双方的最优解既不帮忙也不添乱——它惩罚的只是死守某个固定百分比、不调参的用法。

那么“规模变大”到底改变了什么？把实测数据摊开看，是三推一反。

推力一：失控的基线会一路右移到 80%。 θ_b 不是一个常数，它是“谁在管上下文”的产物。实测里能清楚看到三种活法：

活法	触发点	每步平均背着	一次 2,068 步
程序员手动压（能看见任务）	31%	162.6k	¥7.02
完全不管控	撞 dsh 默认 80%	407.6k	¥16.97
纯按钱调（纸面点：串行不拆分，它站不住）	8%	47.8k	¥3.33

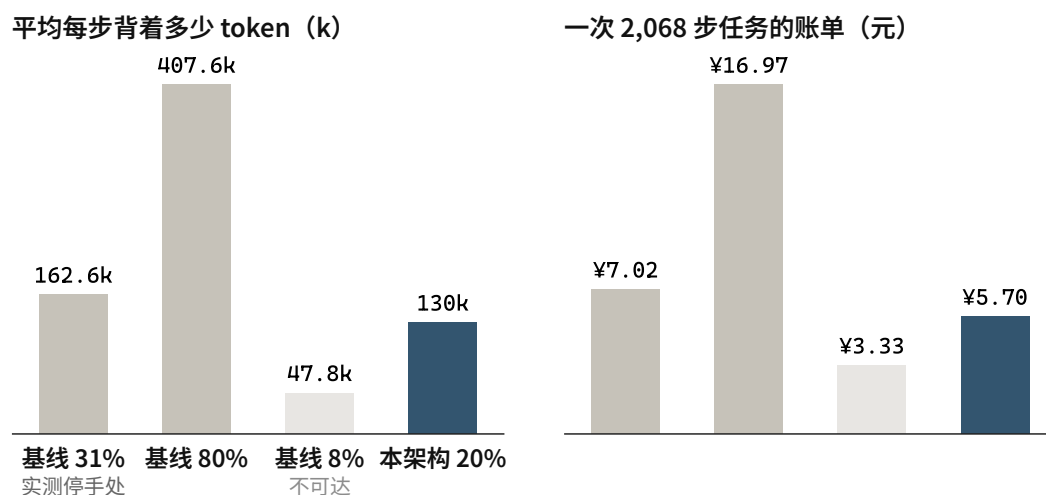


图2 三种活法与本架构的对比。31%一档是实测中人工停手的位置（中位 30.9%）；@8%一档是基线按钱算出的最优点，而串行又不拆分就必须带着足量的信息干活，所以那只是纸面点，实测里也没有人走到；本架构一档取 20% 水位、4 分支、更新量 10k。数据源：附录2 A2.4 / A2.5。

实测里那条**从不压缩、455 步一路顶到 70.2%**的会话，就是“不管控”长什么样；同一批数据里，管与不管的每步成本差 **1.33 倍**。**用户越多、任务越杂、越没人盯上下文，基线就越往右边那一档滑。**

推力二：摘要的容量不随规模增长，而任务必需的上下文会。 实测里残留恒定在 1.3–1.8 万，与任务多大无关——因为它是**模型一次写出来的、长度固定的摘要**。任务一大，被丢掉的细节就多， ρ 就往上走，而 ρ 按**输出价**计费（是背着 的 200 倍）。**这一条是结构性的，不依赖任何参数，也是本架构最硬的论据。**

推力三：用户级的任务天然可并行。 实测 $N = 1$ ——程序员在 2,068 步里一次子代理都没开。用户级任务（多来源、多方案、多文件）天然能切片并行， $N > 1$ 会变成常态。不过 A2.4 已经算过： N 只在**更新量 M 大时才重要**； M 一小， N 就退居次席。

反方向一：我们的残留 $P + s$ 也会随规模增长。 这是唯一的暴露面。基线不管任务多大都只留 1.5 万；我们的前缀随代码库涨、切片随任务面涨。A2.4 给的线是 63k——**所以“前缀能不能比摘要更省地表达全局”，是本架构能不能随规模一起赢的分水岭。** 而这条线管的是**每一轮都构建进上下文、此后被反复读的那些东西**：切片可以按需取用（A2.6 第 6 条），所以真正要长期盯住的其实是前缀 P 。

于是趋势的落点很具体：**不是“任务变长”，而是三件事——基线会不会失控、 ρ 会不会涨、我们的残留会不会跟着涨。** 前两件对我们有利，第三件是风险。

A2.6 没有计入账里的收益

A2.4 的账里有几笔明显对我们有利的钱，**不是在账里被设成了零，就是被按最笨的活法算了一遍。** 把这几笔加回来，就是下面这些——**不在账里计，但应当被知道。**

(1) **平均上下文更短，注意力更集中。** 本架构每一步的平均上下文长度比基线低 Δ （按实测参数：触发点从 31% 降到 20%，平均每步少背约 33k；对完全不管控、一路撞到 80% 的基线，少背约 278k）。上下文越长、模型判断质量越低是可复现的现象。**而更短、更干净、更聚焦的上下文本身就是能力上的更强**——不是同一个模型“少受拖累”，而是同一个模型发挥得更好：同一批问题只给相关部分（约 300 token）优于给全量（约 113k token）^[1]，实测里人停手的位置中位 30.9%，也说明 20%–30% 是能力要求而不是舒适区。

(2) **中段迷失被结构性规避。** 长上下文里，处在中间位置的信息最容易被忽略。切片把工作时的上下文限制在 20%–30%，中间段本身变短，“中间”这个位置不再显著。

(3) **基线必须做一次“丢什么”的判断，我们这边根本没有“丢”这个动作。** 压缩丢的是没有被计划过的信息——你无法预知摘要丢掉的那一行会不会在十步之后被用到，因此不可补偿。而切片只是没有被构建进去：没构建不等于丢了，原文还在 git 里，主 Agent 随时能把它读回来（按需读取），或者干脆把它写进前缀。**压缩这一步必须判断丢什么，而且丢错了无法挽回；分区不需要这个判断——“构建什么”本身就是可以机械核对的。** 这是两条路线最本质的差别，且不依赖任何参数。**这笔差别可以标价。** 基线那九次压缩一共丢掉 194.8 万 token；如果要把丢掉的内容重读一遍、核对摘要是否忠实，按未命中价算是 ¥1.95——相当于它整张账单 ¥7.02 的 28%，而且这笔钱不带来任何收益，只用来让那份摘要变得可信。本架构没有这个动作：它的“校验”就是把该装的内容装出来，**顺带完成，不额外计费。**

(4) **没有有损压缩，因此没有重发现。** 基线的 ρ 来自被摘要丢掉、之后又被需要的细节。本架构里，一个分支做完就退出——它的上下文跟着分支一起消失，但消失的只是“它曾经被构建在上下文里”这件事，不是内容本身（原文仍然留在 git 里，什么时候要就什么时候读）。既然没有任何东西被删掉，也就没有“重新推导一遍”这笔钱。

(5) **探索受约束，路径更收敛。** 分支启动时拿到的那一小段内容（环境视图 + 结构意图映射 + 委派文件）限定了分支的起点与范围，减少盲目通读和随机试错。这同时降低了 T 和 T 的波动幅度——而波动变小在成本上是双份好处：既减少平均返工，也减少做工期估算时必须留的余量。**这一条在 A2.4 的账里被设成了 $\varepsilon = 0$ （协调开销为零），但它的真实符号是负的：**前缀越准，分支越不需要自己找路，总步数 T 反而比基线更小。

(6) **切片不必整段预先构建。** A2.4 的账里，切片 $s = 40k$ 是按“每个分支每一轮都把整段 40k 构建进上下文”算的。但设计本身允许另一条路：只把公共前缀放进去，剩下的文件交给一个工具**按需取用**——没被取用的部分根本不进上下文，于是一路停在台阶四；被取用的部分才走台阶二与台阶三。**这是“从 0 构建”最直接的推论：默认是不构建，构建才是那个需要理由的动作。** 这笔钱能省多少，取决于“分支实际需要取用多少”，我们既没有实测，也不打算把它计进账里，但它说明 A2.4 那个 $P + s = 60k$ 的残留，保守的成分不只是“取大”，还有“假设了最笨的构建方式”。

(7) **更短的上下文直接换成延迟与吞吐。** 本架构每步背着的内容更少，而且每一轮的前缀逐字节相同，于是 prefill 更短、缓存复用更充分。这一层不是我们独有的：^[17] Mooncake 量到前缀缓存命中带来的吞吐与延迟收益；^[37] KVFlow / KVCOMM 在工作流感知的前缀复用

下，多智能体并发仍取得 prefill 与吞吐收益；^[16] Splitwise 给出 prefill 与 decode 的耗时比——1,500 输入 token 的 prefill $\approx$ 6 个输出 token 的 decode。三条都属甲档。这一项按 A2.4 的取法不计入结论（“不拿时间上的好处去补贴成本上的主张”），但不该被当成不存在。

（8）装配可以有多不精准。 A2.4 的两套账之间留着 ¥1.32 的余量（¥7.02 – ¥5.70）。按命中价折算，这笔余量允许本架构每一轮多装约 3.2 万 token 的无关内容——而正常装配是 $P + s = 60k$ ；也就是说，装配里有过半是废料，账上仍然赢。代价的斜率是命中价而不是未命中价，前提是这些废料也必须“稳定地废”（每轮抖动的内容按未命中计价，贵 50 倍）。不过账上容得下不等于注意力容得下：衰减跟的是绝对 token 量，所以上限是 63k 那个预算，不是这 3.2 万。

A2.7 四条例外

前面每一处取法都取在对我们不利的一侧，只有四条例外：两条不是保守取法，而是假设；一条是取法本身越过了真实机制；还有一处是公式值与实测值之间的取舍，方向同样偏在有利的一侧。它们必须单独说清，否则上面的结论会被读得比它实际支持的更强。

一、 $N = 4$ 不是保守值，它把“设计被正确使用”当成了既定事实。 实测是 $N = 1$ ——那条两千步的干净会话里，人一次子代理都没开。在基准档下，这一条把结论从“省 19%”拉到“省 5%”；在前缀按命中计的档位下，从“省 20%”拉到“省 11%”。

二、 $\theta_d = 20\%$ 是能力假设，不是成本结论——而基线那一侧的 8% 连假设都不是。 本节所有“省 19%”都建立在“我们能在 20% 的水位上正常工作”之上，这一条要靠 A2.6 证明，不在算术里。但“两边的最优点都落在 8% 一带”这个巧合，不该被读成“基线也能跑到那里”：基线只有一层上下文，理解与干活挤在同一段内容里；串行又不拆分，它就不可能不带着足量的信息工作。压到 8%，省下的正是它干活所必需的那部分理解——A2.3 已经把那个最优点成立的前提写清了：把 $\rho = 0$ 与“丢掉的东西一文不值”推到极致。所以 8% 对基线是纸面点，不是可选的工作点。

低水位在我们这里为什么成立，是同一条道理的正面。 全量理解在前、拆分在后：理解被编译成那份稳定前缀（短、稳、每一步都在），干活只装自己那一片，缺什么按需读一次。低水位不是设计出来的，是拆分出来的；而拆分的前提是先有一次全量理解。基线没有这两层可分，只有一层，所以它一低就丢能力。

“两边都按在 31% 时我们贵 12%” 因此只是一道算法核对，不是任何一方会选的位置：真要那么跑，等于我们主动放弃唯一的机制优势。

三、前缀整段按未命中，是一条越过了真实机制的取法。 E_d 里的前缀项本该拆成两半：变了的那一段 ΔP 失去缓存，没变的那段继续命中。取 $\Delta P = P$ 等于假装每换一版都要整段重读，差额是 $P(p_m - p_h) = 20k \times (1 - 0.02)/M$ ，也就是 ¥0.0196。

它单独看只占我们总成本的 1.9%（¥0.11 / ¥5.70），看起来无关紧要——但它进的是开方号里的 E_d ，于是被放大成“我们能不能赢下基线自己的最优点”这件事：

取法	E_d	我们的有效残留 R_d	对基线 $R_b = 80.5k$
前缀整段未命中、更新 10k	¥0.060	106.8k	输
前缀按命中、更新 10k	¥0.040	98.4k	输
前缀整段未命中、更新 2k	¥0.028	92.0k	输
前缀按命中、更新 2k	¥0.008	77.5k	赢

单改前缀的计价还不够，必须同时把更新量从保守的 10k 放回现实的 2k， R_d 才能压到 R_b 之下。而这两件事恰好都是这套设计本来就主张的：只写变动的那一段而不是整篇（小 M ）+ 段序稳定（前缀可缓存）。

这里要留意一点：因为每一轮都是从 0 构建，“段序稳定”这句话的分量比它看上去大——它决定的正是“上一轮构建在同一位置的那几段字节，这一轮还在不在同一个位置”。在，就继续命中；不在，就整段按未命中重读一遍。而这份差额的方向是清楚的：把前缀按真实机制计价，只会让我们更便宜。本节“省 19%”用的仍是保守档，没有靠它。

四、压缩间隔取了公式值 202 步，而实测是 230 步——这一条取舍偏在我们有利的一侧。两次压缩之间隔多少步，直接决定基线压多少次、付多少次重置的钱；间隔越长，基线越便宜。表中两个值都给了，我们用了 202。影响很小：基线的重置从 ¥0.26 涨到 ¥0.30，整张账单从 ¥6.99 到 ¥7.02，省的比例从 18.4% 变成 19.0%——0.6 个百分点。但它的方向确实是偏的，所以必须在这里露面：A2.0 的第三条规矩问的正是“这些保守取法是不是真的都在对我们不利”，这一条就是那次复查漏掉的。

第三条不是偏向哪一边，而是越过了真实机制；第四条相反，它偏了方向，但量级已经交代清楚。本节的立场因此是这样：

在能把差值算清楚的所有项目上，我们几乎都取了对自己不利的一侧；四条例外里，三条不是取法问题，而是这套设计能不能兑现的假设——它们的证明不在账上，在 A2.6；剩下一条是压缩间隔取了公式值，偏在有利的一侧 0.6 个百分点，已单独交代。

A2.8 本节结论

不看公式，只看这件事：

1. 主流做法是“把内容都攒在上下文里，装不下了就写摘要、把原文删掉”。删掉的东西以后还要用，就只能重新想一遍——而“想”是按最贵的价格计费的。
2. 这套做法是“每一轮都从空手开始，照着清单去仓库取这一轮要用的东西”。清单上写的是要什么、不是东西是什么，所以写起来很短；取回来的还是原件，用完就放手——它回到的是“不在上下文里”的默认态，而不是被删掉了。
3. 于是困难的问题换了：从“该丢什么”变成“该构建什么”。前者必须由模型做一次判断，判断错了无法挽回（序言里那三次翻车都翻在这里）；后者写的是映射，照着映射就

能核对装对了没有。这套账里省下的钱，大头不是“少写”，而是“根本没构建”。

4. 在这个价位上，“写”极其昂贵：模型写 1 个 token 的钱，等于背着它走 200 步。实测里模型每步只写 1,043 个 token（是背着量的 0.51%），却付出了和“背着”一样多的钱——账单里有四成，就花在这 0.5% 的量上。

看数（一次 2,068 步的真实任务，全部账在 A2.4）：对实测的真实基线省约 19%，对完全不管控的基线省约 66%；对照“基线自己的钱最优”（8%），它比我们便宜 71%，而那一档要求它一个任务压 46 次、实测里人只压了 9 次，而且串行不拆分的结构根本站不住那一档（A2.7 第二条）；“两边都按在 31% 时贵 12%”是算法核对，不是我们会的位置。

成本这一侧，两套架构的账形状相同，差别全在“那条水位线能不能站住”：基线必须带着足量的信息干活，压到它自己的钱最优就等于放弃理解；而我们要站在低水位上，靠的是理解与干活分了两层——先全量理解，再拆分。它在账上表现为 19%，在能力上表现为 A2.6 那八条，其中第一条就是：更短、更干净、更聚焦的上下文本身就是更强。

要把本节从“半定量”推到一张可以直接对照的账单，还差三个数：每轮更新前缀时真正写出的更新量 M 、“前缀 + 切片”的真实残留 $P + s$ 、契约成本的分摊情况（第 1 轮 vs 后续轮）。这三个数要由一次对照实验来量——对照组直接要求完成一个中等规模开源仓库的改造任务，实验组用本架构手动组装前缀并委派。

附录3：引用、外部证据与对照

A3.0 约定

正文与附录2 里的方括号编号（形如 [6]）指向 A3.1 的条目。

先分清三个层次，免得后面对不上：“上下文原生”指的是那一步解耦——Context 从 Harness 里分出来；“Git 树”指的是本文给出的那一处具体实现；附录2 里的数字是这处实现的账。下面凡说“本文的做法”，指的都是第二层。

条目按来源分四档，只有甲、乙两档被用来支撑结论：

档	类型	用法
甲	同行评审论文	可作结论依据
乙	厂商一手文档、官方公告、产品文档	可作机制与计价的依据
丙	预印本（有方法与数据，未经评审）	标明未评审后可用，但不单独支撑结论

档	类型	用法
丁	工程博客、个人基准	只作方向性参考

A3.1 条目

#	来源	档	核验	它支撑什么
[1]	Chroma (Hong, Troynikov, Huber), <i>Context Rot: How Increasing Input Tokens Impacts LLM Performance</i> , 2025-07-14。 trychroma.com/research/context-rot	乙	原文	同一批问题只给相关部分（约 300 token）优于给全量（约 113k token）；18 个模型一致
[2]	NVIDIA (Hsieh 等), <i>RULER</i> , COLM 2024, arXiv:2404.06654	甲	转述	有效上下文只有标称窗口的 16–50%
[3]	Kuratov 等, <i>BABILong</i> , NeurIPS 2024, arXiv:2406.10149	甲	转述	推理任务上的有效利用率约 10–20%
[4]	Zhuang 等, <i>AgentRewind: Recoverable Execution for Long-Horizon LLM Agents</i> , arXiv:2608.14380	丙	转述	从头重来 78.0% > 继续推进 62.2%；只回滚上下文 43.9%，比什么都不做还差
[5]	OpenHands / All Hands AI, <i>Context Condenser</i> 文档	乙	转述	append-only 事件日志 + <code>View</code> 派生视图：状态与提示词是两件事
[6]	Anthropic, <i>How Claude Code uses prompt caching</i> (官方文档)	乙	转述	前缀匹配是精确匹配；同前缀扇出只付一次未命中；压缩会按设计作废整层缓存
[7]	Preble 等, prompt 间共享 token 比例的测量, ICLR 2025, arXiv:2407.00023	甲	转述	85–97% 的 prompt token 与其他请求共享
[8]	Anthropic, Claude Code 官方文档 <i>Create custom subagents</i> , <i>Worktrees</i> , <i>Agent teams</i>	乙	转述	子代理单独提示与工具、worktree 各自打不到缓存；agent teams 约 7× token
[9]	Anthropic (Rajasekaran, Dixon, Ryan, Hadfield 等), <i>Effective context engineering for AI agents</i> , 2025-09-29	乙	原文	最小高信号 token 集；轻量标识符按需载入；子代理只回 1,000–2,000 token 摘要
[10]	LMSYS / SGLang HiCache, 前缀缓存命中率实测 (Claude Code traces over SWE-bench), 2026	丙	转述	同会话后续轮次命中率平均约 98%
[11]	vLLM × AgentX, 生产轨迹统计, 2026-09-08	丙	转述	每轮中位输入 142K、输出 444、命中 >96%、44% 会话含子代理
[12]	Red Hat AI Americas, vLLM v0.19.0 KV 缓存字节稳定性验证, github.com/redhat-ai-americas/memory-hub	丁	转述	前缀追加 41 个字符后命中率 98.29%→1.78%
[13]	Liu 等, <i>Lost in the Middle</i> , TACL 12:157–173, 2024	甲	转述	相关信息落在中段时表现显著下降

#	来源	档	核验	它支撑什么
[14]	Cognition (Walden Yan), <i>Don't Build Multi-Agents</i> , 2025-06-12	乙	原文	并行写者脆弱; 子代理宜只答不写
[15]	Kim 等, <i>Towards a Science of Scaling Agent Systems</i> , arXiv:2512.08296	丙	转述	独立代理放大错误 17.2×, 集中式 4.4×; 顺序推理任务退化 39–70%
[16]	Splitwise (Patel 等), ISCA 2024, arXiv:2311.18677	甲	转述	1,500 输入 token 的 prefill $\approx$ 6 个输出 token 的 decode
[17]	Mooncake (Moonshot AI / 清华), FAST 2025 最佳论文, arXiv:2407.00079	甲	转述	前缀缓存命中带来的吞吐与延迟收益
[18]	DeepSeek, <i>Models & Pricing</i> (官方定价页)	乙	原文	缓存命中 ¥0.02 / 未命中 ¥1 / 输出 ¥4 (空闲时段, 高峰 2 倍)
[19]	DeepSeek, <i>DeepSeek-V4.1-Flash</i> 发布公告, 2026-09-10	乙	转述	“Cache-hit charges often account for a large share of agent costs”
[20]	Bai, Huang, Wang, Sun, Mihalcea, Brynjolfsson, Pentland, Pei, <i>How Do AI Agents Spend Your Money?</i> , arXiv:2604.22750	丙	转述	8 模型 $\times$ 500 个 SWE-bench 任务: 输出单价约 80 $\times$ 于缓存读, 但因体量, 缓存读仍是最大成本项
[21]	Manus, <i>Context Engineering for AI Agents</i> , 2025-07-18	乙	转述	平均输入输出比约 100:1; KV 缓存命中率是生产 agent 最重要的单一指标
[22]	Microsoft Learn / Azure OpenAI Foundry, <i>Prompt caching</i> (更新至 2026-08-11)	乙	转述	GPT-5.6 起缓存写入另计费; 首个 1,024 token 必须完全一致
[23]	Lindenbauer 等 (JetBrains Research / TUM), <i>The Complexity Trap</i> , NeurIPS 2025 DL4Code, arXiv:2508.21433	甲	转述	观察遮罩与 LLM 摘要能力打平, 成本约减半
[24]	Cursor (Cassano, Rush), <i>Training Composer for longer horizons</i> , 2026-03-17	乙	转述	自摘要约 1,000 token (基线 >5,000), 压缩错误降 50%
[25]	Cognition (Walden Yan), <i>Multi-Agents: What's Actually Working</i> , 2026-04-22	乙	原文	写保持单线程、其他代理只贡献智力; 不共享上下文的审查更有效
[26]	Augment Code, <i>AI Coding Cost Analysis</i> , 2026-07-24	丁	转述	每步 token -75%, 但轮数 4 $\rightarrow$ 14, 总量只 -14%; 缓存削不掉二次项
[27]	<i>Governance Decay</i> : 压缩静默抹除治理约束, arXiv:2606.22528	丙	转述	7 个模型家族违规率 0% $\rightarrow$ 30%, 个别 59%
[28]	Zhang 等 (Stanford / SambaNova / UC Berkeley), <i>Agentic Context Engineering</i> (ACE), ICLR 2026, arXiv:2510.04618	甲	原文	brevity bias 与 context collapse: 迭代重写会侵蚀细节
[29]	Ehrlich, Blackman, <i>LCM: Lossless Context Management</i> , arXiv:2605.04050	丙	原文	分层摘要 DAG + 指向每一份原文的无损指针

#	来源	档	核验	它支撑什么
[30]	Conner, <i>pi-fold: Agent-Governed Lossless Context Folding</i> , Zenodo, DOI 10.5281/zenodo.21856874	丙	原文	SHA-256 地址库 + peek / expand 逐字节取回；成本 $0.55\times$ 于原生压缩（单作者，每臂一次运行）
[31]	Packer 等, <i>MemGPT: Towards LLMs as Operating Systems</i> , arXiv:2310.08560	丙	转述	虚拟上下文分页：主上下文 + recall/archival 外部存储
[32]	Santoni, <i>Contextual Memory Virtualisation</i> , arXiv:2602.22402	丙	转述	会话史当版本化状态（DAG + 快照/分支/裁剪）；裁剪的经济性依赖 prompt caching
[33]	Hsu, Lu, <i>GRACE: Scoped Verification for Reliable Long-Horizon Agentic Context Evolution</i> , arXiv:2607.09175	丙	转述	结构化限定范围的更新 $\text{pass}^3 = 0.673$ ，平铺整体重写只有 0.191
[34]	Laban, Hayashi, Zhou, Neville (Microsoft), <i>LLMs Get Lost In Multi-Turn Conversation</i> , ICLR 2026, arXiv:2505.06120	甲	转述	多轮相比单轮平均掉 39%
[35]	Cursor (Wilson Lin), <i>Scaling long-running autonomous coding</i> , 2026-01-14	乙	转述	20 个并行 agent 塌缩成 2-3 个的吞吐，最终删掉协调角色
[36]	Anthropic (Nicholas Carlini), <i>Building a C compiler with a team of parallel Claudes</i> , 2026-02-05	乙	转述	16 个并行 agent 会互相覆盖，靠解耦与锁文件收敛
[37]	KVFlow / KVCOMM, 多智能体工作流的前缀复用测量, NeurIPS 2025, neurips.cc/virtual/2025/loc/san-diego/poster/119883	甲	转述	工作流感知的前缀复用收益
[38]	Anthropic (Hadfield 等), <i>How we built our multi-agent research system</i> , 2025-06-13	乙	原文	多智能体比对话多约 $15\times$ token；并行收益与任务类型强相关
[39]	akitaonrails, <i>llm-coding-benchmark</i> , 手动编排对照报告	丁	原文	同一批执行者被给定边界后 46→93、71→94、87→97；Opus 单独一局里缓存读占账单 60%
[40]	DeepSeek, <i>DeepSeek Harness</i> (官方开源仓库 deepseek-ai/deepseek-harness, 口号 “Everything is a Plugin”, v0.1)	乙	原文	本文实测与计价所依据的 Harness (dsh) 本身
[41]	Xia, Deng, Dunn, Zhang, <i>Agentless: Demystifying LLM-based Software Engineering Agents</i> , FSE 2025, arXiv:2407.01489	甲	原文	三阶段（本地化→修复→校验）、不让模型决定未来动作、不用复杂工具，SWE-bench Lite 32.00%（96 个正确修复）、\$0.70，胜过所有开源软件 agent
[42]	Gutiérrez, Shu, Gu, Yasunaga, Su, <i>HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models</i> , NeurIPS 2024, arXiv:2405.14831	甲	原文	单步取回即可媲美或超过 IRCoT 这类迭代检索，且便宜 10-30 倍、快 6-13 倍

#	来源	档	核验	它支撑什么
[43]	Kang, Chen, Han, Inan, Wutschitz, Chen, Sim, Rajmohan, <i>ACON: Optimizing Context Compression for Long-horizon LLM Agents</i> , ICML 2026, arXiv:2510.00615	甲	原文	峰值 token 降 26–54%、任务成功率高于已有压缩基线——“把上下文压得更狠”这条路线目前最强的一篇
[44]	Xue, <i>When and How Context Rot Appears in Coding Agents: A White-Box Study of Agent Skills in Code Auditing</i> , arXiv:2607.17937, 2026-08-01	丙	原文	详细的外部清单 10/10 通过、泛泛的自检只有 5/10 ($p=0.0325$)；干净子代理只在开头挡得住继承来的杂乱，自身轨迹一长问题就在本地复发；onset 不是稳定阈值
[45]	Mei, Yao, Ge 等, <i>A Survey of Context Engineering for Large Language Models</i> , arXiv:2507.13334, 2025-07	丙	原文	把上下文工程立为一门学科，其分类中 Context Management 独立成节（基础约束 / 记忆层级与存储 / 上下文压缩）
[46]	Anthropic, <i>Building effective agents</i> , 2024-12-19（官方工程文档）	乙	原文	“最成功的实现用的是简单、可组合的模式，而不是复杂框架”；编排者—工作者等模式
[47]	Anthropic, Claude 平台文档 <i>Context editing</i>	乙	原文	可选择性清除工具结果；并明确提示 清除会作废缓存前缀 ，要先算够不够本（ <code>clear_at_least</code> ）
[48]	Anthropic, Claude 平台文档 <i>Memory tool</i>	乙	原文	把学到的东西写进 memory 目录、跨会话取用，“ 不必把一切都留在上下文窗口里 ”，支持 just-in-time 取回
[49]	LangChain, <i>Context Management for Deep Agents</i> （工程博客）	丁	原文	大工具结果与旧的写入参数 卸载到文件系统、按需检索取回 ；卸载到无可卸载时才摘要
[50]	LangChain, <i>Organizing Context in a Multi-Agent Harness</i> , 2026-09-08（工程博客）	丁	原文	子代理=并行推理+上下文隔离、在 fresh context 里干活；而 fork 继承父会话更便宜（吃前缀缓存）——两条路都写明
[51]	Li, Li, Zhang, Mei, Bendersky, <i>Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach</i> , EMNLP 2024（Industry Track）	甲	原文	资源充足时长上下文平均优于 RAG，但 RAG 成本显著更低——与 Chroma 的 focused/full 结论条件不同、方向相反
[52]	jianzhichun/permafrost, <i>Real Claude Code e2e 实测记录</i> （工程博客）	丁	原文	DeepSeek 按渲染后的请求（tools→system→messages）缓存；缓存身份还包含客户端 HTTP 头指纹与请求参数；工具列表顺序变动使命中率 71%→33%；跨会话预热不可能

A3.2 相似架构对比与区分

有几项工作与本文看起来高度相似，必须逐条说清分界，否则“内容离开上下文也不丢”这层会被误读成本文的贡献。

这张表要读成一张分区图：它们绝大多数是在“累积下来的上下文里做减法”，本文换掉的是信源。

工作	它的机制	与本文相同的那一步	与本文的分界
MemGPT / Letta ^[31]	主上下文 + recall/archival 外部存储；被逐出的消息写回外部存储，用时按语义检索取回	内容离开上下文不等于失去内容	外部存储放的是 模型自己写的记忆与消息历史 ，取回靠语义检索；且它是 累积的 ，本文每轮 重投影
LCM ^[29]	分层摘要 DAG 压缩旧消息，但保留 指向每一份原文的无损指针	无损指针，摘要只作索引	它保的是 对话的原文指针 ；本文不保留对话，只保留 环境 ，上下文由环境重新推导
pi-fold ^[30]	完成的片段原地折叠成简报，原文进 SHA-256 地址库，peek / expand 逐字节哈希校验取回	工程上最接近：字节级无损取回	它维护一条 追加式线历史 （作者自述 76 次纯追加 + 6 次批重写）；本文每轮 从零重装
CMV ^[32]	会话史当版本化状态，DAG + 快照/分支/裁剪，机械剥掉工具输出等膨胀	版本化、结构无损、保留全部消息	它裁剪的是 累积日志 ；本文的日志不存在。它并明确指出裁剪之所以划算 依赖 prompt caching ——而这正是本文要正面处理的那一条
OpenHands ^[5]	append-only 事件日志，递给模型的是由 View 派生的视图	状态 ≠ 提示词，提示词是派生视图	它的视图是对日志的 过滤 ；本文的视图是对环境的 重装
Anthropic 的按需载入 + 子代理 ^{[8][9]}	轻量标识符（路径、查询、链接）运行期载入；子代理只回 1,000–2,000 token 摘要	写地址而非正文 ；子代理只回果实	它的地址用于 运行期取用 ，压缩仍是第一杠杆；本文每轮重建前缀，压缩不再是杠杆
主流压缩 + 机械清除 ^{[6][22][23]}	累加到阈值后摘要，或按类型机械清除旧工具结果	都承认机械操作能打平模型判断	它们仍在 同一份累积上下文里做减法 ；本文把“该丢什么”换成“该装配什么”
ACON ^[43]	用任务无关的压缩器把长历史压成更短、可迁移的表示，按峰值 token 与成功率联合优化	都承认长轨迹的上下文必须被处理	它把力气全花在“怎么压得更狠、更省”上（峰值 token -26–54%）；本文 取消压缩这个动作本身 ——内容默认不构建进去，原文留在环境里，要用的时候读一次
LangChain Deep Agents ^[49]	大工具结果与旧的写入参数卸载到文件系统，按需检索取回；卸载到无可卸载时才摘要	卸载 + 按需取回 ，摘要垫底	它仍是一份 持续推进的累积历史 ，卸载只是把字节从上下文里搬走；本文每一轮从环境重装，历史本身不存在

分界线只有一条：信源。 上表每一项都要求“**对话历史被保存下来**”——它们的无损，是无损地指回那份历史。本文要求的是“**环境是权威源，对话可以丢弃**”，所以上下文不是被裁剪出来的，而是被**重装**出来的。这一条差别决定了两件事：一是不需要另建一个原文库（git 已经是），二是“每轮从零重装”才成立——它们不能重建，因为没有可重建的权威源。

A3.3 主张与外部证据

本文主张	外部证据	关系
上下文越长，判断越不可靠	Chroma ^[1] : 18 个模型，只变长度不变难度，只给相关部分（约 300 token）一致优于给全量（约 113k token）	支撑，是“少背就是能力”最直接的一手实验
衰减跟的是 绝对 token 量 ，不是窗口百分比	RULER ^[2] 有效窗口为标称的 16–50%，只有约一半模型在 32K 上保住质量；BABILong ^[3] 推理任务上“只有效利用 10–20%”	支撑；本文的“20–30%”是夹在这两条实测带之间的经验值
背着是账单最大的一笔	Bai et al. ^[20] : 输出单价约 80× 于缓存读，但因累积上下文体量， 缓存读仍是最大成本项 ；DeepSeek 发布公告 ^[19] : “Cache-hit charges often account for a large share of agent costs”；Manus ^[21] : 平均输入输出比约 100:1	支撑，三源独立
写比读贵得多	Splitwise ^[16] : 1,500 输入 token 的 prefill 与 6 个输出 token 的 decode 耗时相同；Anthropic 与 OpenAI 对缓存 写入 收 $1.25 \times -2 \times$ ^[22]	支撑；本文按 硬件耗时 比取这一条，不按单价之比
高命中率是常态	LMSYS ^[10] 约 98%；vLLM × AgentX ^[11] 命中 >96%；Red Hat ^[12] 字节稳定前缀 98.29%；Preble ^[7] 85–97% 的 prompt token 与其他请求共享	支撑，四条独立来源包住本文实测的 98.6–99.7%
共享前缀在系统层面确有收益	KVFlow / KVCOMM ^[37] : 工作流感知的前缀复用，在多智能体并发下仍取得吞吐与 prefill 收益	支撑正文“前缀缓存利用率高”与“装配时段序稳定”两处的判断
子代理只把果实交回主干	Anthropic ^[9] : 子代理 “returns only a condensed, distilled summary (often 1,000-2,000 tokens)”；Claude Code ^[8] 示例：读入 6,100 token、交回 420 token；Cursor ^[24] 自摘要约 1,000 token 对基线 >5,000	支撑，且与本文实测“每步写 1,043 token”同量级
边界的价值大于执行者本身	akitaonrails ^[39] : 同一批执行者被给定边界后 46→93、71→94、87→97；归因“执行者写了每一行，但规划者决定了边界”	支撑（丁档，只作方向性参考）
上下文不该累积	AgentRewind ^[4] : 从头重来 78.0% > 继续推进 62.2%；OpenHands ^[5] 以事件日志 + 派生视图工作	支撑；AgentRewind 为丙档预印本
回滚需要三件套	AgentRewind ^[4] 消融：只回滚上下文 43.9%（比什么都不做还差）、只回滚环境 51.2%、三者齐备 87.8%	支撑本文“分支 + 环境 + 信号回流”缺一不可
机械操作能打平模型判断	Complexity Trap ^[23] : 观察遮罩与 LLM 摘要在能力上打平、成本约减半	支撑；且序言记录过更早的一手经验（工具剪切中间层，实测收益极低后搁置）
压缩有账外代价	Augment ^[26] : 每步 -75%，但轮数 4→14、总量只 -14%；Governance Decay ^[27] : 压缩后治理约束被静默抹掉，违规率 0%→30%	支撑，均为丙/丁档，只作方向性参考
长会话里“累积”本身在退化	Laban et al. ^[34] : 多轮相比单轮平均掉 39%，且走错之后不会自己走回来	支撑重建与回滚

本文主张	外部证据	关系
边界的价值独立于上下文长度	Cognition ^[25] : 代码审查代理在 与写作者完全不共享上下文 时效果最好	支撑上下文隔离
不让模型决定下一步的固定阶段, 比自由探索更强也更便宜	Agentless ^[41] : 三阶段 (本地化→修复→校验)、不让模型决定未来动作、不用复杂工具, SWE-bench Lite 32.00% (96 个正确修复)、 \$0.70 , 胜过所有开源软件 agent	支撑, 甲档 ; 与上一条“边界的价值大于执行者本身”是同一方向的甲档证据, 本文的“契约+断言验收”也落在这一侧
一次取回优于反复迭代	HippoRAG ^[42] : 单步取回即可媲美或超过 IRCot 这类迭代检索, 而 便宜 10-30 倍、快 6-13 倍	支撑, 甲档 ; 对应“按需取用”与 A2.6 第 6 条
外部清单/契约验收优于模型自检	Context Rot in Coding Agents ^[44] : 详细的外部清单 10/10 通过, 泛泛的自检只有 5/10 (p=0.0325)	支撑, 丙档; 对应“合并前的断言与测试”
干净上下文的好处有 明确时效边界	同上 ^[44] : 干净子代理只在开头挡得住继承来的杂乱, 一旦它自己的轨迹变长, 同一个问题会在本地复发; 且 onset 不是稳定阈值	双向 : 支撑“种子”, 同时给种子的有效期划界——这正是本文把轮长压到“一轮一分支”的理由, 不是可选项
上下文工程已是一门独立学科	Mei 等 ^[45] : 把 Context Engineering 立为学科, 分类中 Context Management 独立成节 (基础约束 / 记忆层级与存储 / 上下文压缩)	支撑, 丙档; 对应正文“Context 成为一等公民”
“简单可组合”优于复杂框架	Anthropic ^[46] : 最成功的实现用的是简单、可组合的模式, 而不是复杂框架	支撑, 乙档; 对应正文“以工程复杂度换取落地价值”
工业界官方口径已转向“让工具结果离开上下文”	Claude 文档: context editing 可选择性清除工具结果, 并提示 清除会作废缓存前缀、要先算够本 ^[47] ; memory tool 把知识写进 memory 目录、“不必把一切都留在上下文窗口里”、支持 just-in-time 取回 ^[48] ; LangChain 把大工具结果卸载到文件系统、按需取回, 卸载不动了才摘要 ^[49]	支撑, 乙/丁档; 但三者都是在累积式上下文里做减法, 与本文“不构建”的分界见 A3.2

A3.4 已经发生过的坑，与本文对应的设计区分点

下面每一条都是同类系统已经踩过的坑，以及本文在此处做了哪一步不同的选择。这些坑连起来，就是这条路能用与不能用的边界。

已经发生过的坑	谁遇到的	本文对应的区分点
子代理有自己的系统提示与工具集, 打不到父会话的缓存 ; 只有精确继承的 fork 才行	Claude Code ^{[6][8]}	种子必须是 共享前缀的严格延伸 : 公共部分逐字节同一, 分支差异只能落在前缀之后 (正文“装配时段序稳定”)
缓存按机器与目录分家, 同一仓库的不同 worktree 互相打不到	Claude Code ^[6]	环境隔离与前缀解耦: 环境标识绝不进前缀
前缀追加 41 个字符 , 命中率 98.29%→ 1.78% , 官方口径“Effectively a complete miss”	Red Hat ^[12]	完全前缀组装 + 段序稳定 (同前; 附录2 A2.7 的 ΔP)

已经发生过的坑	谁遇到的	本文对应的区分点
工具列表的顺序也是缓存身份，MCP 列表一变动命中率 71%→33%	社区实测 ^[52]	前缀内不放任何会漂移的字节
并发写者互相覆盖；20 个并行 agent 塌缩成 2-3 个的吞吐，最后把协调角色删掉	Anthropic C 编译器 ^[36] 、Cursor ^[35]	各写各的虚拟文件 + 事前契约 + 单写者合并 （正文“作用于分支的虚拟文件”与“规避并行写者 N ² 的冲突”）
独立代理把错误放大 17.2 倍 ，集中式只有 4.4 倍；顺序推理任务退化 39-70%	Kim et al. ^[15]	本文不是“独立代理投票”：主 Agent 定契约与断言、单写者合并—— 正落在该文的正结果一侧（集中式协调）
多智能体比对话贵 15 倍 ^[38] ，agent teams 约 7 倍 ^[8]	Anthropic 自己的数字	那 15× 来自 每个子代理各背一份上下文 ；共享前缀 + 按需取用打的正是这一点
让模型重写自己上一轮写的正文，会“context collapse”、越写越薄	ACE ^[28] ；GRACE ^[33] 对照里结构化更新 0.673 对平铺重写 0.191	本文不让模型平铺重写正文：模型只重写该更新的那一段，原文留在环境里、要用的时候读一次—— 正落在 GRACE 结构化更新的那一侧
压缩会 主动作废 整层缓存（官方口径：“invalidates the conversation layer”）	Claude Code ^[6]	重建 ≠ 重排：段序稳定时前缀照旧命中
预算充足时，长上下文一致优于检索增强	Li et al. ^[51] (EMNLP 2024, 甲档)	与 Chroma ^[1] 的 focused/full 结论方向相反，两文条件不同、互相抵消；本文的主张不是“任何条件下小上下文都更强”，而是 “把理解与干活分成两层之后，更短、更聚焦的上下文更好，而且不必因此丢内容”
子代理另起一份提示能隔离上下文，但吃不到父会话的缓存；fork 继承整段对话虽然吃得到缓存，却把上一轮的杂乱一起背进来	LangChain 的 harness 实践把这两条路都写明了 ^[50] （缓存那一侧的一手口径见本条第一行 ^{[6][8]} ）	同第一条：种子取的是中间那条路—— 共享前缀的严格延伸 （既不另起一份而丢掉缓存，也不继承整段对话而背进杂乱）

A3.5 至今没有人做过的对照

这一栏划出引用管不到、只能靠实验回答的部分。它要测的是解耦本身，不是某处实现的好坏。

一、把“机械 vs 模型”控制住之后，再比“累积 vs 重建”。现有文献里，机械清除^[23]与重建架构从未在同一条件下同台。对照应当是四臂，且 B 与 C' 必须使用同一种现成的机械手段，否则又是拿“有”比“没有”：

臂	内容	它单独回答什么
A	累积 + LLM 压缩	现行默认
B	累积 + 厂商级现成的机械清除	机械能否打平模型判断
C	每轮从环境重建 + 共享前缀	机械清除在 重建 架构下的收益（与 A→B 的差对照）

臂 内容	它单独回答什么
C' 每轮从环境重建 + 共享前缀 + 同一种机械清除	本文的完整形态

A 对 B 切出来的是本文与别人共享的那部分原则；B 对 C' 切出来的才是本文自己的增量——这两臂机械手段完全相同，唯一剩下的变量是“上下文是累积的还是每轮重建的”。

二、“压缩税”没有任何直接测量。现有文献里没有任何来源把“模型写摘要”这一笔单独计入账；最接近的净额数据是 Augment 的“每步 -75%、总量 -14%”^[26]，而 ACON^[43] 报的也是峰值 token 降 26-54% 这个总账，没有单列压缩本身的开销。所以本文把它算成一张账单，是本文自己的贡献，引用只能提供方向性支持，不能提供数值背书。

三、“前缀更准 ⇒ 更少工具调用”这条因果没有任何论文测过。附录2 A2.6 第 5 条把它写成设计意图是对的；但它至今是待测项。

四、“N² 写冲突”是本文的用词。冲突现象被反复观察到 (A3.4)，但没有文献这样命名或量化它。

A3.6 口径与适用范围

项	附录2 的说法	适用范围
三档价格 ¥0.02 / ¥1 / ¥4 ^[18]	按 DeepSeek 计价	这是 DeepSeek-V4.1-Flash 空闲时段的官方价；高峰为 2 倍；取低谷价是保守取法
未命中是命中的 50 倍、输出是命中的 200 倍	两个比值决定整笔账的形状	该比值 只对 Flash 成立 ：Pro 是 30×/90×；Anthropic 与 OpenAI 的缓存读为输入价的 0.1× (10×) ^[22] ；Bai et al. 的价格表约为 80× ^[20] 。 不能讲成 prompt caching 的普遍性质
背着 38.5% / 首读 21.7% / 写 39.8%	三条实测会话的账单构成	这是在 DeepSeek 价目 + 长会话稳定前缀 这一工作量上测得的。换到 Anthropic 文档里的 agent 会话，按其用量重算约为 背着 51.0% / 缓存写入 33.9% / 输出 14.4% ^[22] —— 换价目只会让“背着”占比更高 。所以它是工作量形状，不是 agent 的普遍形状
“背着的便宜被体积抵消”	背着单价极低但数量巨大	更准确： 价格上便宜，资源上不便宜 。按公开的 KV 尺寸，1M token 的缓存在 Llama-3-8B 一类模型上约 122 GB；命中价约 \$0.0028/M ^[18] 。命中价是 算力价（省掉的 prefill） ，不是存储价
一次重置的未命中“只付一次，不按分支数付”	本架构重置成本构成	前提比原句更严：缓存要求 完整前缀单元匹配 且为 best-effort；分叉点必须落在同一个已存单元边界上。这是 设计纪律 ——正文“装配时段序稳定”与“公共部分逐字节相同”正是为它写的
三条会话遥测 (9,023 步 / 203,660 token 每步 / 98.6-99.7% 命中)	参数全部来自实测	这是 本文自己的一手遥测 ，外部无法复核。其中 203,660 token/步属于“长会话 + 工具密集 + 含子代理”的区间 (vLLM × AgentX 的生产中位是每轮输入 142K ^[11])， 不是编码 agent 的普遍水平 ；98.6-99.7% 的命中率则有四条独立来源包住 (A3.3)
“人在 30% 上下停手”	九次人工压缩的中位 30.9%	这是 本文自己的观察 ，不是行业规范；公开默认值要激进得多 (Claude Code 的自动压缩在窗口的 83.5% 上下触发) ^[6]

项	附录2 的说法	适用范围
前缀逐字节相同就能命中缓存	装配时段序稳定	前缀字节相同是 必要不充分 ：实测 DeepSeek 的缓存身份还包含客户端的 HTTP 头指纹与请求参数 ——字节完全相同、只换一套请求头（或改了 <code>max_tokens</code> ）会掉到 0% ；另外分支之间共享可以命中， 跨会话预热不可能 （请求必须完整匹配一个已持久化的前缀单元） ^[52]
模型每轮重写需要模型更新的那一段	更新量 M 远小于整篇前缀	映射是版本相关的 ：写它的那一轮过去了，一次合并就会让结构整体位移。本文 没有规定它绑定到哪一版 （绑到提交哈希，或照 ^[30] 的 SHA-256 地址库存取，都是可行做法），也 没有给出“写全了没有”的校验 ——能校验的是“装回来的字节与原文一致”，不能校验的是“该写进映射的有没有漏”。这一条是设计缺口，不是保守取法

A3.7 未采信与不可用清单

- **有一处说法只见于媒体转述，本文未加引用**：那句“Deepseek 甚至将这个理念写入到了他们的招聘要求中”即属此类——可核到的只是“该公式被行业广泛使用”与“DeepSeek 已组建 Harness 团队并公开招募相关岗位”。
- **不引用**：CompactionRL (arXiv:2607.05378) 与 Recursive Language Models (arXiv:2512.24601) ——未核到原文。
- **不引二手数字**：ACE 的“18,282 → 122 token”collapse 数字来自第三方笔记而非论文正文，本文只引其摘要原话^[28]。
- **同行评审比例**：甲档共 14 条，其余为乙档厂商一手文档与丙、丁档预印本、工程博客。
- **【转述】的含义**：原始页面或论文正文未能直接访问，数字经镜像、索引或第三方转述核实；方向可信，精度待复核。
- **本版新增的 12 条** (^[41]^[52]) **全部以【原文】核验**：逐条打开原始页面或论文正文，表里写的即读到的。其中^[44]是白盒实测、^[49]^[50]^[52]是工程博客与实测记录，只作方向性参考。
- **^[10]^[11] 缺可核标识，只作方向性参考**：它们只有名称与日期，没有 URL 或编号（^[10]是 LMSYS / SGLang HiCache 的实测、^[11]是 vLLM × AgentX 的生产轨迹统计）；^[12]、^[37]本版已补上可核位置。它们支撑的结论另有^[7]^[12]两条可核来源包住。
- **A2.0 的第三条规矩复查漏了一处，本版补记**：压缩间隔 k_b 取了公式值 202 步而不是实测的 230 步，方向偏在有利的一侧（0.6 个百分点），已作为 A2.7 的第四条例外单独列出。

A3.8 一句话

本文的价格模型与“背着是账单最大的一笔”有官方口径、独立研究与厂商原话三重支撑；“无损可回取”这一层已有先行工作，本文的区别在信源是环境而不是对话历史；而本文真正的成本主张——压缩税——至今没有被任何人直接测量过，所以它是本文自己的账，不是引文的结论。把“机械 vs 模型”与“累积 vs 重建”分开的那张四臂对照，也还没有人做过。

但“还没有人做过对照”不等于“没有支持”。这套设计的每一步单独拿出来，都已经甲、乙档证据：不让模型决定下一步的固定阶段胜过自由探索^[41]，单步取回胜过反复迭代^[42]，外部清单胜过模型自检^[44]，厂商的官方口径也已经转向“让工具结果离开上下文”^[47]^[48]。本文缺的不是支持，是把这些支持拼成“每轮重建”的那一次对照。

版式说明

单文件离线版 · 文字与 Markdown 逐字一致 · 版本 f3c38cb · 2026-09-15